

Programming with Spatial Data

Contents

1. About	1
2. Welcome	3
2.1 Why Learn to Program?	3
2.2 Is This the Right Course for You?	4
2.3 How to Approach This Course?	5
2.4 Why Programming Still Matters	5
2.5 Is Programming Hard?	6
2.6 Why Python?	6
2.7 Why Programming with Geospatial Data?	6
3. Team	9
3.1 Contact	9
3.2 MS Teams	9
4. Syllabus	11
5. Schedule	13
5.1 Holidays Schedule	13
5.2 Project Schedule	13
6. Error Culture	16
6.1 Learning Through Productive Struggle	16
6.2 Asking Questions and Discussing Errors	16
7. Use of AI	18
7.1 Use of AI in this course	18
8. Resources	20
8.1 Online Courses	20
8.2 Python Books	20
8.3 YouTube	21
8.4 Podcasts	21
8.5 Blogs	21
8.6 Others	22
9. Acknowledgments	24
10. Setup	26
10.1 1. Introduction	26
10.2 2. Learning Objectives	26
10.3 3. Essential Software Tools	27
10.4 4. Tool Integration and Workflow	30
10.5 5. Running Code Examples	30
10.6 6. Key Takeaways	31
11. Conda	33
11.1 1. Introduction	33
11.2 2. Learning Objectives	33
11.3 3. Installing Conda	33
11.4 4. Understanding Conda Concepts	36
11.5 5. Working with Conda	38
11.6 6. Working with uv	43
11.7 7. Best practices	45

11.8	8. Exercises	46
12.	VS Code	49
12.1	1. Introduction	49
12.2	2. Learning Objectives	49
12.3	3. Installing VS Code	49
12.4	4. Essential Extensions	50
12.5	5. Connecting VS Code to Conda	51
12.6	6. Working with VS Code	52
12.7	7. Exercises	55
13.	Git	58
13.1	1. Introduction	58
13.2	2. Learning Objectives	58
13.3	3. Git vs. GitHub	58
13.4	4. Git Setup	59
13.5	5. Understanding Git	59
13.6	6. Working with Git	61
13.7	7. Exercises	64
14.	Colab	67
14.1	1. Introduction	67
14.2	2. Learning Objectives	67
14.3	3. Getting Started	67
14.4	4. The Notebook Environment	67
14.5	5. Managing Python Packages	68
14.6	6. Working with Files and Data	69
14.7	8. Running Course Notebooks	70
14.8	9. Exercises	70
15.	JupyterLab	73
15.1	1. Introduction	73
15.2	2. Learning Objectives	73
15.3	3. Getting Started	73
15.4	4. Launching JupyterLab	74
15.5	5. Notebooks as Scientific Documents	75
15.6	6. Productivity Fundamentals	77
15.7	7. Working Without a Local Setup	78
15.8	8. Exercises	78
16.	Motivation	82
16.1	1. From data to understanding	82
16.2	2. Why visualization matters	82
16.3	3. Space and time together	83
16.4	4. Why programming helps	83
16.5	5. What this course aims to do	84
16.6	6. Summary	84
17.	Glossary	86

1. About

Introduction to SDS210



This section provides the **big-picture context** for *Programming with Spatial Data (SDS210)*.

Before you start writing code, it is important to understand **what this course is about**, **who it is designed for**, and **how to approach it successfully**. The pages in this section explain the motivation behind the course, the learning philosophy, and the expectations placed on you as a student.

In particular, this section helps you to:

- understand **why programming matters** in spatial data science,
- see **how Python fits** into modern geospatial workflows,
- clarify **what kind of learning experience** to expect,
- know **how to work with errors, questions, and AI tools** productively,
- and get to know the **teaching team and communication channels**.

You do not need to read everything in this section in one go.

Instead, treat it as a **reference point** that you can return to throughout the semester whenever you want to better understand *why* the course is structured the way it is.

The following subsections cover:

- the course motivation and learning objectives,
- guidance on workload, expectations, and study strategies,
- perspectives on programming, Python, and geospatial data,
- error culture and responsible use of AI,
- team, contact information, and organisational details.

Together, they define the **conceptual and pedagogical framework** within which all technical content of the course is embedded.

2. Welcome

Welcome to *Programming with Spatial Data* (ESS341, GEO876, SDS210). This course introduces **computational thinking and Python programming for geospatial applications**. In an increasingly data-driven and automated world, the ability to design, write, and organise code for spatial analysis has become an essential skill for addressing environmental, social, and urban challenges.

Rather than focusing on isolated tools or button-clicking workflows, this course emphasises **thinking with code**. You will learn how to translate spatial questions into structured, reproducible workflows and how to implement these workflows using Python. Over the semester, you will build a solid foundation in core programming concepts and progressively apply them with widely used geospatial libraries such as GeoPandas, RasterIO, and Matplotlib. Throughout the course, strong emphasis is placed on writing **clear, structured, and well-documented code**, supported by Jupyter Notebooks and Git to enable transparent and reproducible work.

The course concludes with an individual programming project in which you design and implement your own solution to a spatial data problem. By the end of the course, you will be well prepared to take on more advanced challenges in spatial data science, both in research and in applied contexts.

2.1 Why Learn to Program?

You might be wondering why you should even bother learning to program at all. After all, you are probably not a computer scientist, and neither are we. Yet all authors of this course use programming regularly as part of their scientific and applied work. Often, the programs we write are short, simple, and highly practical but they make a real difference.

The reason is simple: **geography and geoscience have become increasingly quantitative**. We now work with larger datasets, repeated observations, numerical models, and automated analyses. Many of these tasks quickly become inefficient, error-prone, or impossible when done manually. Even basic programming skills allow you to explore data more flexibly, apply analyses consistently, and document exactly what you did.

Programming does not replace GIS software, but it **extends it**. Many tools you already know, such as QGIS, are built around Python and allow you to automate workflows, inspect data programmatically, or develop custom tools. Understanding a bit of code helps you move beyond button-clicking and better understand what happens behind the scenes.

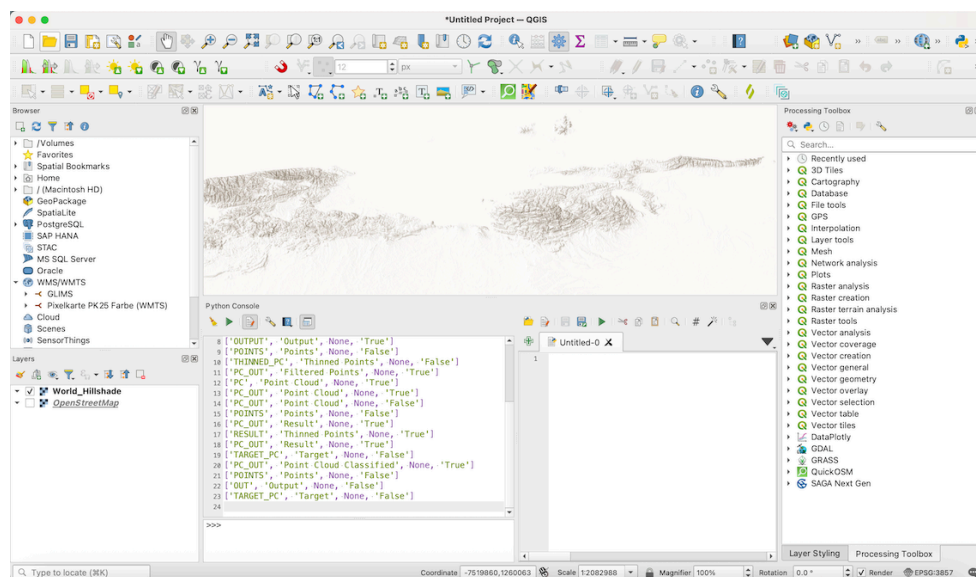


Figure 2.1: Via the Python console you can interact with GIS data in the [QGIS](#) program.

Programming can also be **surprisingly enjoyable**. Once you learn the basics, complex problems can be broken down into smaller, logical steps. You can test ideas immediately, adjust your approach, and see whether your solution works. This problem-solving mindset is one reason why programming is increasingly introduced early in education. The aim is not to train programmers, but to train **thinkers**.

Finally, programming supports **good scientific practice**. The **scientific method** relies on transparency, testing, and reproducibility. Writing code forces you to be explicit about your assumptions, your steps, and your decisions. Data processing, analysis, and visualisation become repeatable rather than ad-hoc. Throughout this course, you will learn programming practices that help make your work reproducible, communicable, and reusable, which are key principles of modern, **open science**.

2.2 Is This the Right Course for You?

2.2.1 Target Audience

Programming with Spatial Data is designed for students who want to learn how to work with spatial data using Python and who are at an early stage in their programming journey.

The course is particularly well suited for:

- students with little or no prior programming experience,
- students who have encountered code before but lack confidence or routine,
- students from geography, environmental sciences, or related fields who want to move beyond graphical user interfaces and better understand what happens behind the scenes.

No advanced programming background is assumed. The course starts from first principles and gradually builds up complexity. At the same time, it takes the learning process seriously: you will not only learn *how* to write code, but also *why* certain approaches work, how to structure solutions clearly, and how individual decisions affect reproducibility and maintainability.

This is not a course about pushing buttons. It is about using logic, programming, and analytical thinking to tackle real-world spatial problems in a transparent, reproducible, and communicable way.

2.2.2 Learning Objectives

This course aims to build a solid foundation in Python programming for spatial data analysis. Rather than focusing on short-term solutions or isolated tools, the emphasis is on developing transferable skills that help you plan, implement, and explain code-based workflows.

By the end of the course, you should be able to work independently on small to medium-sized spatial programming tasks and understand how individual coding decisions fit into a larger analytical process.

On successful completion of the course, you will be able to:

- apply core Python programming concepts such as variables, loops, functions, and data structures,
 - design a conceptual workflow before implementing a coding solution,
 - write clear, modular, and reproducible Python code with appropriate documentation,
 - read, understand, and debug existing Python code for spatial analysis,
 - design and implement object-oriented solutions to read, manipulate, visualise, and export spatial data.
-

2.3 How to Approach This Course?

This course follows a flipped classroom approach that combines self-paced learning with interactive, in-person lab sessions. Each week, you are expected to engage with the lecture videos and the accompanying written material before attending the lab. These resources introduce the programming concepts and spatial data techniques that form the basis for the practical work.

The lab sessions are used to apply what you have learned, clarify open questions, and work collaboratively on coding exercises. Instructors are present to discuss your code, help you reason through problems, and support you when you encounter difficulties during implementation.

This structure allows you to:

- learn new concepts at your own pace through videos and readings,
- deepen your understanding during labs through hands-on coding and peer exchange,
- receive targeted support where and when you need it.

Because the classroom is flipped, preparation is essential. Except for the first week, you are expected to come to each lab having already engaged with the assigned materials. The labs are designed for applying, testing, and discussing ideas, not for first exposure to new content.

2.3.1 Workload and Commitment

This module is demanding and requires consistent effort throughout the semester. Success does not come from last-minute preparation, but from regular practice, careful reading, and repeated experimentation with code.

According to UZH guidelines, a 5 ECTS module corresponds to approximately 150 hours of total workload. With about two hours of timetabled lab activity per week, this leaves several hours per week for self-study. In practice, you should expect to spend time almost every day reading, coding, and practicing.

2.3.2 Why the Effort Is Worthwhile

There is a growing demand for analysts, planners, and geographers who can think computationally and work confidently with data and code. Across research, public administration, industry, and non-profit organisations, spatial data skills combined with programming literacy are increasingly sought after.

By investing the time and effort required in this course, you are developing skills that are widely applicable and highly valued. Learning to reason with code opens up opportunities across many domains and allows you to engage more critically and creatively with spatial data.

2.4 Why Programming Still Matters

With recent advances in AI, it is reasonable to ask whether learning to program is still worth the effort. If machines can generate code, why spend time learning Python yourself?

In the following short video, the speaker argues that programming remains a high-impact skill not because of syntax, but because it teaches problem solving, systems thinking, and the ability to turn ideas into working solutions. These are precisely the skills this course aims to develop.

The video is not about mastering a specific language or tool. Instead, it provides a broader perspective on why programming knowledge continues to matter, even in an increasingly automated world. Focus on first three minutes to get this point.

2.5 Is Programming Hard?

Many students begin this course wondering whether programming is too difficult, whether advanced mathematics are required, or whether they are “the right type of person” to learn how to code.

In the following short video, these concerns are addressed directly. Learning to program is not effortless, but it is far more accessible than many people assume. What matters most is persistence, curiosity, and having a meaningful reason to continue when things become challenging.

2.6 Why Python?

Python is the programming language used throughout this course. It is widely adopted across science, data analysis, and applied domains because it combines expressive power with readability and a rich ecosystem of libraries. Python allows you to focus on problem solving, data workflows, and reproducibility rather than on technical overhead. For this course, Python provides the best balance between ease of entry and long-term relevance for spatial data science workflows.

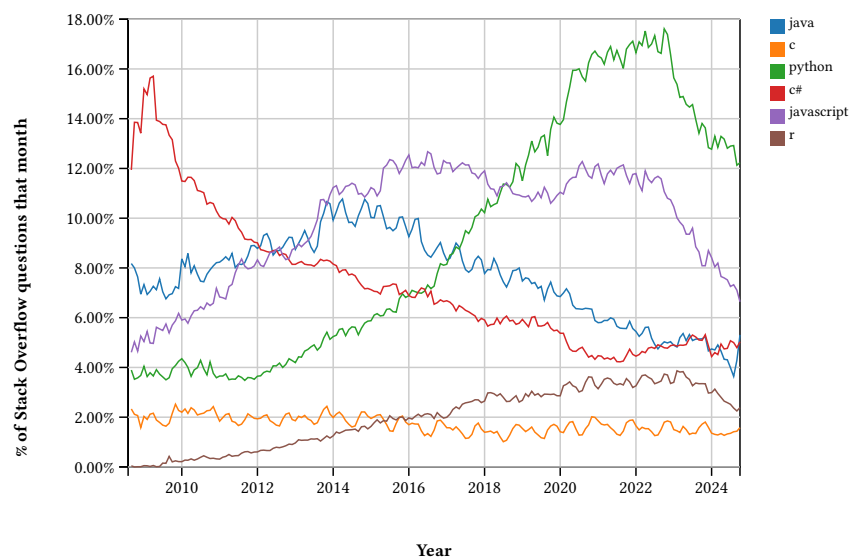


Figure 2.2: The popularity of programming languages as indicated by the number of questions on Stack Overflow.

In the following video, the speaker highlights why Python is often chosen for data-driven work and why it is particularly suitable for beginners who want to build practical, transferable skills.

2.7 Why Programming with Geospatial Data?

Geospatial data is complex by nature. It is often large, multi-dimensional, linked to coordinate reference systems, and collected repeatedly over time. While graphical tools are useful for exploration and visualisation, they quickly reach their limits when analyses become complex, repetitive, or need to be reproduced.

Programming allows you to work with geospatial data in a transparent and controlled way. Instead of manually repeating the same steps, you can describe entire workflows explicitly and apply them consistently across datasets, regions, or time periods. Code documents not only what you did, but also how and why you did it, supporting reproducibility, collaboration, and critical evaluation.

In this course, programming is used as a complement to GIS software. The goal is not to replace graphical tools, but to extend them with methods that provide flexibility, transparency, and analytical depth when working with real-world spatial data.

3. Team

Our amazing team of instructors consists of the following people (in alphabetical order). We will support you on your learning path during the lab exercises and on Teams.

3.1 Contact

Questions about course content or exercises

For questions related to the course material (e.g. “How do I resolve problem X in task Y?”), please use the *respective Microsoft Teams channel*. This way helps everyone benefit from the discussion.

To help us support you efficiently, always include:

- a brief description of the problem
- the exact error message (if any)
- code snippet or a screenshot
- a Colab or GitHub link (when applicable)

Personal or administrative questions

For personal matters or course-related questions specific to SDS210, ESS342 or GEO876, please contact the module coordinator: hendrik.wulf@geo.uzh.ch

Office hours (during the semester): Tuesdays and Wednesdays, 14:00–15:00 in room Y25-K-12

3.2 MS Teams

Teams is the central communication platform for this course. It is used to:

- share announcements and course information
- ask and answer questions
- submit the final project report

To ensure consistent communication, OLAT is used only to provide this course syllabus. All other course-related communication takes place on MS Teams. Please ensure that Teams notifications for channels and assignments are on (via profile settings).

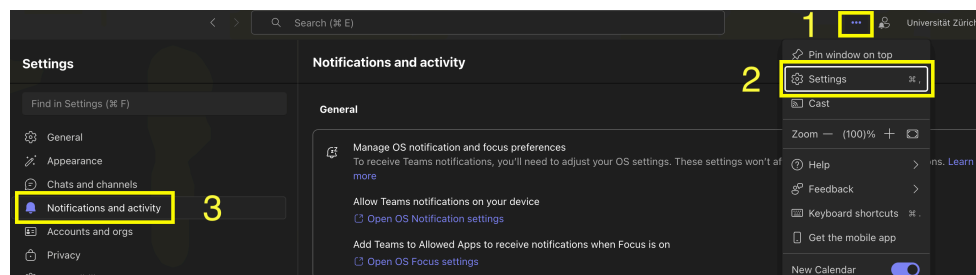


Figure 3.1: Teams notifications settings.

If you need help outside of lab hours, please post your question in the appropriate lab channel on Teams. All students are encouraged to respond and help where possible. This shared discussion space benefits everyone and reduces duplicate questions.

4. Syllabus

For quick reference, the course syllabus is embedded below. It can also be found on OLAT and Teams and you can also open it in a new tab if needed.

5. Schedule

The course schedule is shown in two ways: by holidays, and by project dates.

Labs: Thursdays 16:15 – 18:00, Fridays 10:15 – 12:00, and 12:15 – 13:45 in [Y25-J-08/09/10](#).

5.1 Holidays Schedule

W	L	Thu	Fri	Holidays	Topic
1	1	19 Feb	20 Feb		Getting Started with Python
2	2	26 Feb	27 Feb		Variables & Data Types
3	3	05 Mar	06 Mar		Loops and Conditional Statements
4	4	12 Mar	13 Mar		Understanding and Defining Functions
5	5	19 Mar	20 Mar		Using Libraries and Web APIs
6	6	26 Mar	27 Mar		Working with DataFrames
7	7	02 Apr	03 Apr	Good Friday	Analysing Vector Data
8	•	09 Apr	10 Apr	Easter Break	
9	8	16 Apr	17 Apr		Visualizing Spatial Data
10	9	23 Apr	24 Apr		Arrays and Raster Data
11	10	30 Apr	01 May	May Day	Data Cubes and Time Series
12	11	07 May	08 May		Object-Oriented Programming
13	12	14 May	15 May	Ascension Day	Reproducibility and Best Practices
14	•	21 May	22 May		Project Feedback
15	•	28 May	29 May		Project Presentation Week

Table 5.1: The marked dates indicate public holidays, when no labs take place. W = Week. L = Lesson.

5.2 Project Schedule

W	L	Thu	Fri	Project	Topic
1	1	19.02	20.02		Getting Started with Python
2	2	26.02	27.02		Variables & Data Types
3	3	05.03	06.03		Loops and Conditional Statements
4	4	12.03	13.03		Understanding and Defining Functions
5	5	19.03	20.03		HUsing Libraries and Web APIs
6	6	26.03	27.03		Working with DataFrames
7	7	02.04	03.04		Analysing Vector Data
8	•	09.04	10.04	—	Easter Break
9	8	16.04	17.04	Projects Introduction	Visualizing Spatial Data
10	9	23.04	24.04		Arrays and Raster Data
11	10	30.04	01.05		Data Cubes and Time Series
12	11	07.05	08.05		Object-Oriented Programming
13	12	14.05	15.05		PReducibility and Best Practices
14	•	21.05	22.05	Project Report Submission	Project Feedback

15	•	28.05	29.05	Code Discussion and Justification	Project Presentation Week
----	---	-------	-------	-----------------------------------	---------------------------

Table 5.2: The marked weeks indicate events related to the course projects. W = Week. L = Lesson.

6. Error Culture

Errors are a normal and essential part of learning anything, especially in a programming-focused course. Writing code that does not work as expected is not a failure. It is a key mechanism through which understanding develops.

Error messages, unexpected results, and broken scripts provide valuable information about how Python interprets your instructions and where assumptions or mental models do not yet match the behaviour of the code.

Learning to read, interpret, and resolve errors is therefore a core learning objective of this course.

If you notice any mistakes or areas for improvement in these learning materials, please share your [feedback](#) as helps us to refine and improve the materials for everyone.

6.1 Learning Through Productive Struggle

During the course you will encounter syntax errors, confusing error messages, and code that only partially works. This is expected.

Working through these situations helps you develop practical skills in debugging, reasoning about code, and identifying the source of a problem. These skills are transferable and essential for working with real data and real systems, where solutions are rarely straightforward.

Progress in programming often comes in small steps, through testing, adjusting, and trying again.

6.2 Asking Questions and Discussing Errors

If something does not work or does not make sense, you are encouraged to ask questions during labs or via Teams. Discussing errors openly helps everyone learn and reduces the impression that others “got it right” on the first try.

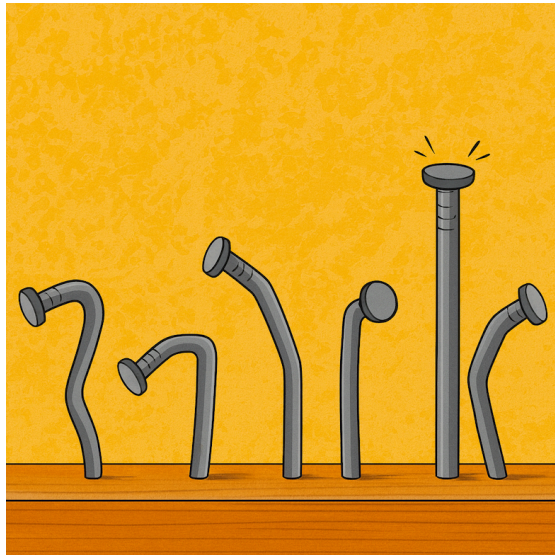


Figure 6.1: *Errors are part of the shared learning experience and discussing them is both expected and valued*

7. Use of AI

7.1 Use of AI in this course

AI tools such as large language models (LLMs) can be valuable companions when learning Python programming, **if used intentionally and responsibly**.

In this course, you may use AI tools (e.g. Claude, Gemini, [Copilot](#), ChatGPT) to:

- clarify Python syntax and concepts,
- debug errors and understand error messages,
- explore alternative implementations or approaches,
- improve code readability and documentation.

Used well, AI can help you learn faster and work more efficiently.

7.1.1 AI Is a Learning Aid, Not a Replacement

The goal of this course is not to produce working code as quickly as possible, but to develop:

- a solid understanding of **Python programming concepts**,
- the ability to **reason about code**,
- confidence in **explaining and justifying your solutions**.

If an AI tool provides a complete solution that you cannot explain or adapt, it is **not helping your learning**.

AI should *support* your thinking, not replace it.

7.1.2 Expectations for This Course

When using AI tools, you are expected to:

- remain actively involved in the problem-solving process,
- understand and be able to explain all submitted code,
- treat AI-generated output as **suggestions**, not authority,
- critically review, test, and adapt any AI-assisted code.

You are responsible for everything you submit, regardless of whether AI was involved.

7.1.3 Transparency and Responsibility

Using AI is allowed in this course, but **uncritical copy-paste is not**.

If AI played a substantial role in your solution, be prepared to explain:

- what you asked the tool,
- what you changed,
- and why the final solution works.

This approach reflects real-world programming practice, where understanding and accountability matter more than speed. Experienced people share [this view](#).

8. Resources

Learning **Spatial Data Science (SDS)** is a bit like learning a new language *and* how to use it to solve real-world problems.

The resources below support different stages of your learning journey, whether you want to **strengthen your Python basics**, **learn geospatial workflows**, or **think more deeply about spatial data science**.

All resources listed here are:

- free and openly available
 - widely used in university teaching
 - well aligned with the SDS mindset (practical, reproducible, problem-oriented)
-

8.1 Online Courses

These courses work well alongside this course. You don't need to complete any of them, **pick what matches your current level and interests**.

- [Geo-Python](#)

Kamyar Hasanzadeh & Dave Whipp

A hands-on course focusing on Python for geospatial data analysis and workflows.

- [Introduction to GIS Programming](#)

Qiusheng Wu

Combines GIS concepts with modern Python-based spatial analysis.

- [Foundations of Spatial Data Science](#)

Jon Reades

Strong focus on concepts, theory, and how spatial data science fits into the broader data science landscape.

- [Python Foundation for Spatial Analysis](#)

Ujaval Gandhi

A very solid introduction to Python using spatial examples. Ideal if you are new to Python or want to refresh the basics.

8.2 Python Books

These books work well as **companions**. Use them to look things up, revisit concepts, or read selected chapters when something is unclear.

UZH students also have access to a large collection of Python and data science books via the [O'Reilly for Higher Education platform](#) provided by the University Library.

- [Introduction to Python for Geographic Data Analysis](#)

Henrikki Tenkanen, Vuokko Heikinheimo, David Whipp

A geo-focused introduction to Python programming and geographic data analysis, written specifically for students and practitioners working with spatial data.

- [Geographic Data Science with Python](#)

Sergio J. Rey, Dani Arribas-Bel, Levi J. Wolf

A comprehensive introduction to geographic data science, covering methods, theory, and tools for analysing spatial data in modern, data-rich environments.

- [The Python Coding Book](#)

Stephen Gruppeta

Clear explanations and practical examples for learning Python effectively.

- [Think Python](#)

Allen B. Downey

An accessible introduction to Python for beginners, especially useful if you are new to programming or want a clear conceptual foundation.

8.3 YouTube

Short video playlists are ideal for **visual explanations**, **tool walkthroughs**, and **quick refreshers**. Use them selectively when you want to *see* how something is done.

- [Geospatial Python](#)

Matt Forrest

A YouTube playlist focused on learning Python for GIS and geospatial workflows.

- [Introduction to GIS Programming](#)

Qiusheng Wu

A video-based introduction to GIS programming concepts and Python-based workflows.

- [Python Foundation for Spatial Analysis](#)

Ujaval Gandhi

Companion videos to the *Python Foundation for Spatial Analysis* course.

- [Getting Started with Visual Studio Code](#)

Microsoft

Short, practical tutorials covering essential VS Code features such as editing, IntelliSense, the command palette, terminals, and extensions.

8.4 Podcasts

Podcasts are great for **big-picture thinking**, **community insights**, and staying connected to the wider spatial and Python ecosystem. Perfect for commuting or low-effort learning.

- [Talk Python to Me](#)

Host: Michael Kennedy

A weekly podcast covering Python libraries, tools, and people from the Python community. (Link to [Spotify](#) & [Apple Podcasts](#))

- [Spatial Stack](#)

Host: Matt Forrest

Conversations on modern geospatial technology, AI, big data, and real-world spatial projects.

(Link to [Spotify](#) & [Apple Podcasts](#))

- [Geomob Podcast](#)

Hosts: Ed Freyfolge, Steven Feldman, Alastair Graham, Denise McKenzie

A community-driven geospatial podcast linked to the Geomob event series across Europe. Focuses on geoinnovation, industry perspectives, and informal discussions.

(Link to [Spotify](#) & [Apple Podcasts](#))

- [The MapScaping Podcast](#)

Host: Daniel O'Donohue

Weekly interviews and discussions on GIS, remote sensing, earth observation, and digital geography.

(Link to [Spotify](#) & [Apple Podcasts](#))

8.5 Blogs

Blogs are ideal for **short-form insights**, **current debates**, and **practical tips** that often do not (yet) appear in textbooks or courses. They are especially useful for staying up to date with **emerging tools**, **methods**, and **perspectives** in spatial data science.

- [Spatial Thoughts](#)

Practical tutorials, reflections, and insights on geospatial workflows, Python, and spatial data science by Ujaval Gandhi.

- [GeoAI Unpacked](#)

A Substack exploring the intersection of geospatial data, machine learning, and AI, with accessible explanations and applied examples.

- [Strategic Geospatial](#)

Industry-focused blog covering trends, applications, and strategic developments in the geospatial sector.

8.6 Others

The following platforms are not structured courses, but they are extremely useful for **quick lookups**, **syntax clarification**, and **solving specific coding problems**.

Use them as reference tools when you need a fast answer or an alternative explanation.

- [W3Schools – Python Tutorial](#) A concise, example-driven reference for Python syntax and basic programming concepts. Particularly useful for quickly checking how a specific function, method, or data type works.
 - [Stack Overflow](#) A large community-driven question-and-answer platform for programmers. Ideal for troubleshooting error messages, understanding unexpected behaviour, and exploring alternative implementations.
-

These resources are by no means complete or perfect. If you have any suggestions for additional resources or criticism, please [let us know](#).

9. Acknowledgments

This course builds on the work from various contributors listed in the [resources](#) section, who have generously made their (teaching) materials and knowledge openly available. In particular, I used resources from [Ujaval Gandhi](#), [Qiusheng Wu](#) and the teaching teams of [GEO876](#) & [ESS341](#), who laid the foundations for this course.

Furthermore, this course benefited from the support of the [GIUZ](#), the [MNF Teaching Hub](#), and [UZH Continuing Education](#).



Figure 9.1: Credit where credit is due. Without these guys, we wouldn't be using Python in this course. Source: [Wikipedia](#)

Finally, a big shoutout to the [Jupyter Book team](#) for developing a community-driven tool that enables the creation of reproducible, accessible, and hopefully engaging educational content.



10. Setup

Overview of Software Tools



10.1 1. Introduction

Knowing Python is important, but it's only part of the story. To work effectively with spatial data, you also benefit from a **set of tools that work well together**.

Think of geospatial programming like building something complex: one tool alone won't get you very far. You need different tools for different jobs, like: managing software, writing and testing code, keeping track of changes, exploring data, and running analyses on your own computer or in the cloud.

In this section, you'll get a **guided overview of the core tools** used in modern geospatial programming workflows. You won't be expected to master them yet. The goal is to understand:

- what each tool is used for,
- how the tools fit together, and
- why this setup supports reproducible and collaborative work.

This overview will help you **build a mental map** of the workflow you can build throughout this course. In the following lessons and labs, you can gradually start using these tools in practice, one step at a time.

10.2 2. Learning Objectives

After working through this section, you should be able to:

- **Describe what the main software tools are used for** in geospatial programming.
- **Explain how these tools work together** in a typical spatial data analysis workflow.
- **Choose suitable tools for a given task** and explain your choice in simple terms.

Don't worry if this feels abstract at first. These learning goals are about **understanding roles and workflows**, not about mastering every tool right away. You'll revisit and apply them throughout the course as you start using the tools in practice.

10.3 3. Essential Software Tools

Modern geospatial programming is not built around a single tool, but around a **small ecosystem of tools**, each with a clear role in the workflow. In this section, you'll get an overview of six core tools you'll encounter in Spatial Data Science.

You don't need to learn all of them at once. For now, focus on **what each tool is for** and **where it fits** in the bigger picture.



10.3.1 Package Management: Conda

10.3.1.1 What it does

[Conda](#) manages your Python setup *and* the libraries your code depends on, especially the tricky geospatial ones.

10.3.1.2 Think of it like this

Conda is your project environment manager. Instead of having one messy Python installation for everything, Conda lets you create clean, isolated environments. One environment per project or lab.

10.3.1.3 Why it matters

Geospatial libraries (e.g. for reading rasters or projections) rely on system-level components that are difficult to install correctly. Conda takes care of these dependencies for you and helps avoid the classic: *"It works on my laptop, but not on yours."*

10.3.1.4 What you gain

- One environment per project (no version conflicts)
 - Reliable installation of complex geospatial libraries
 - Reproducible setups across different machines and operating systems
-



10.3.2 Code Development: VS Code

10.3.2.1 What it does

[VS Code](#) is your **main workspace** for writing, reading, and debugging code.

10.3.2.2 Think of it like this

VS Code is your control centre. It's where code, notebooks, version control, and debugging come together in one interface.

10.3.2.3 Why it matters

As projects grow, plain text editors quickly become limiting. VS Code helps you write cleaner code, catch mistakes early, and understand what your program is doing, which is especially important when working with spatial data and larger scripts.

10.3.2.4 What you gain

- Smart code completion and helpful error messages
- Integrated debugging tools to step through your code
- Support for both Python scripts and Jupyter notebooks
- Built-in Git tools for tracking changes

- A rich ecosystem of extensions for geospatial work
-



10.3.3 Version Control: Git

10.3.3.1 What it does

[Git](#) keeps track of changes in your files over time and records how your project evolves.

10.3.3.2 Think of it like this

Git is your time machine and safety net. You can go back to earlier versions, compare changes, and try new ideas without risking your working code.

10.3.3.3 Why it matters

Geospatial projects often grow quickly and involve many files, datasets, and experiments. Git helps you stay organised, avoid losing work, and collaborate with others in a structured and transparent way.

10.3.3.4 What you gain

- A complete history of your project
 - The ability to experiment safely and undo mistakes
 - Clear collaboration workflows when working with others
 - A professional standard used in research and industry
 - Easy sharing and backup through platforms like GitHub
-



10.3.4 Cloud Computing: Colab

10.3.4.1 What it does

[Colab](#) lets you run Jupyter notebooks in your web browser using cloud based computing resources.

10.3.4.2 Think of it like this

Colab is a ready to use computer in the cloud. You open a notebook and start coding without installing anything on your own machine.

10.3.4.3 Why it matters

Setting up geospatial software can be time consuming and hardware intensive. Colab removes these barriers by giving you instant access to a working environment that runs on powerful remote machines.

10.3.4.4 What you gain

- Immediate access with no local installation
 - Computing power beyond your own laptop
 - Pre installed Python and geospatial libraries
 - Simple sharing and collaboration through links
 - A practical option for working with large datasets
-

10.3.5 Interactive Analysis: JupyterLab

10.3.5.1 What it does

[JupyterLab](#) is an interactive environment where you can run code, inspect results, create visualisations, and document your thinking in one place.

10.3.5.2 Think of it like this

JupyterLab is your digital lab notebook. It lets you mix code, maps, plots, and explanations while you explore and refine your analysis.

10.3.5.3 Why it matters

Spatial data analysis is rarely linear. You try something, look at the result, adjust your approach, and try again. JupyterLab supports this iterative way of working and makes your reasoning visible and reproducible.

10.3.5.4 What you gain

- Code, figures, and explanations in a single document
- A flexible workspace for working with multiple notebooks and files
- Strong support for interactive data exploration
- Easy access to different data formats and sources
- An extensible environment for specialised geospatial tools



10.3.6 Containerization: Docker

10.3.6.1 What it does

[Docker](#) packages an entire software environment into a container. This includes the operating system, Python, libraries, and your code.

10.3.6.2 Think of it like this

Docker is a sealed box that contains everything your project needs to run. If the box runs on one computer, it will run the same way on another.

10.3.6.3 Why it matters

Geospatial software often depends on complex system libraries that are hard to install consistently. Docker removes this uncertainty and makes sure your analysis behaves the same across different machines and setups.

10.3.6.4 What you gain

- Identical environments across different computers
- Simple distribution of complex software setups
- Isolation between projects to avoid conflicts
- Strong support for reproducible research
- A clear path from development to deployment

Takeaway:

Each tool solves a different problem. Together, they form a workflow that supports clean code, reproducibility, and collaboration. You'll start simple and gradually build up this toolkit as the course progresses.

10.4 4. Tool Integration and Workflow

The real power of these tools comes from **using them together**. Each tool has a clear role, but none of them stands alone. Think of this section as a map of how everything connects.

This workflow can be used in any spatial programming project.

10.4.1 A typical development workflow

1. **Conda** sets up your project environment and installs the required libraries
2. **VS Code** is where you write, run, and debug your code
3. **Git** records changes and keeps your work organised over time
4. **JupyterLab** supports interactive exploration, visualisation, and explanation
5. **Google Colab** lets you run notebooks in the cloud when setup or hardware is a limitation
6. **Docker** makes sure the whole setup can be reproduced anywhere

Each tool solves a specific problem. Together, they support clean code, experimentation, and reproducibility.

10.4.2 How collaboration fits in

In practice, your workflow often looks like this:

1. Write and structure code in VS Code
2. Explore data and results in Jupyter notebooks
3. Track changes and versions with Git
4. Share code and notebooks through platforms like GitHub or Colab
5. Run the same environment on different machines using Docker
6. Keep everything consistent with Conda environments

10.5 5. Running Code Examples

You can run all code examples from this Jupyter Book in different ways. Choose the option that best fits your situation and experience level.

10.5.1 Cloud options

If your goal is to read and try examples quickly, use a cloud option. These options work directly in your browser and require no local installation.

- **Google Colab**

Runs notebooks on cloud computers and is ideal if you want more computing power or an easy way to share your work.

<https://colab.research.google.com/github/HendrikWulf/sds210-jb/blob/main>

- **Binder**

Opens the book examples in a temporary online environment. This is a good choice for a quick look or first exploration.

<https://mybinder.org/v2/gh/HendrikWulf/sds210-jb/HEAD>

10.5.2 Local options

These options run on your own machine and give you full control over your setup.

- **Conda environment**

Uses a local Python environment managed by Conda. You will learn how to set this up step by step in the next sections.

- **Docker**

Starts a ready to use environment that behaves the same on any computer. This is useful if you want a reliable and reproducible setup.

This Docker option is not yet set up for this course, but will be implemented later on.

10.6 6. Key Takeaways

Here are the main ideas to keep in mind as you move on to the next sections.

Think in workflows, not tools

The real strength of this setup comes from combining tools. Each one solves a specific problem, but together they form a complete and reliable workflow.

Learn progressively

You do not need to master everything at once. Start with the basics like Conda and VS Code. Add more tools as your projects grow in size and complexity.

Reproducibility matters

Managing environments, tracking changes, and sharing setups are essential parts of professional geospatial programming. They help others understand, run, and trust your work.

Stay flexible

Local tools and cloud platforms each have their place. Being comfortable with both allows you to work effectively in different situations and on different machines.

You are learning professional practice

These tools reflect how geospatial programming is done in research and industry. The skills you build here are transferable and long lasting.

11. Conda

Introduction to Python Package Management

11.1 1. Introduction

One of the biggest challenges in Python programming is not writing code, but **setting up a working environment**. This is especially true in data science and geospatial analysis, where projects often rely on many libraries that need to work together.

Different projects may require different versions of the same package. Some geospatial libraries depend on additional system software for things like coordinate transformations or reading spatial file formats. When these pieces do not match, things break quickly.

This is where **package management** becomes essential.

Package managers help you install the right software, keep projects separated, and make sure your code runs the same way on different computers. Instead of fighting with installations, you can focus on analysis and problem solving.

11.1.1 Why package management matters

- Geospatial libraries rely on complex system dependencies
- Different projects need different package versions
- Isolated environments prevent conflicts between projects
- Reproducible environments make results reliable
- Shared environment files support collaboration

In this course, we introduce **Conda**, which is well suited for geospatial software because it can manage both Python packages and system libraries. You will also get to know **uv**, a very fast tool for managing Python packages when system level dependencies are not required.

The goal of this section is not to memorise commands, but to understand **how to create reliable and reproducible environments**. These skills will save you time, reduce frustration, and support professional geospatial programming workflows throughout your Spatial Data Science journey.

11.2 2. Learning Objectives

After working through this section, you should be able to:

- **Explain why package management and isolated environments matter** for geospatial programming.
 - **Create and manage project specific environments** using Conda.
 - **Choose appropriate tools and practices** to build reproducible and shareable Python environments, including when to use uv.
-

11.3 3. Installing Conda

11.3.1 Miniconda or Anaconda

You can choose between:

- **Miniconda**: a minimal installation that includes only Conda (~50–100 MB)
- **Anaconda**: a much larger distribution that comes with many preinstalled packages (~3–5 GB)

For this course, **Miniconda is recommended**. It is lightweight, transparent, and encourages you to install only what you actually need.

Both Miniconda and Anaconda can be installed either via:

- a **command line installer**, or
- a **graphical installer**

Here, we focus on the **command line installation**, as it works reliably across systems and helps you better understand how your Python environment is set up. If you feel uncertain about using the shell, you may want to briefly review the [basic commands](#) and [networking](#) section in this [Bash Tutorial](#) by W3Schools before continuing.

Official installation guides (recommended):

- Miniconda: <https://www.anaconda.com/docs/getting-started/miniconda/install>
- Anaconda: <https://www.anaconda.com/docs/getting-started/anaconda/install>

Take your time with the installation. A clean and well understood setup will make the rest of the course much smoother. If you encounter any issues while installing [Miniconda](#) or the [Anaconda Distribution](#), please refer to their website links for troubleshooting assistance.

Follow the steps below to install **Miniconda** using the command line. This approach is reliable, transparent, and works consistently across systems.

Windows

Open **Windows PowerShell** (not the regular Command Prompt) and paste the three commands below there.

To open PowerShell, press Windows key and type **PowerShell**

These three commands quickly and quietly download the latest 64-bit Windows installer, rename it to a shorter file name, perform a silent install, and then delete the installer:

```
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Windows-x86_64.exe -o .\miniconda.exe
start /wait "" .\miniconda.exe /S
del .\miniconda.exe
```

After installing, close the PowerShell and open Anaconda Prompt to use Miniconda.

macOS (Apple Silicon)

Step 1: Download and install Miniconda

Run the following commands **line by line** in the Terminal.

These commands will:

- create a directory called miniconda3 in your home directory (-p flag stands for parents)
- download the Miniconda installer script to your new miniconda3 directory
- install Miniconda in silent mode (-b), taking care of updates (-u), in the given path (-p)
- remove the installer script after installation

```
mkdir -p ~/miniconda3
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-arm64.sh -o ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

Step 2: Activate Miniconda

After installation, **close and reopen your Terminal**, or run:

```
source ~/miniconda3/bin/activate
```

You should now see (base) at the beginning of your command prompt.

Step 3: Initialize conda for your shell

Initialize conda for all currently available shells so it works automatically in new terminal sessions:

```
conda init --all
```

Step 4: Verify your installation

Run any conda command. For example: `conda list` (displays a list of packages installed in your active environment and their versions) or `conda --version` (displays conda's version number).

```
(base) name@computer ~ % conda --version
conda 25.11.1
```

macOS (Intel)

As of August 15, 2025, Anaconda has stopped building packages for Intel Mac computers (osx-64). Existing Intel (MacOSX-x86_64) installers are still available at <https://repo.anaconda.com/miniconda/> and the last Miniconda installer release for Intel Mac computers will be 25.7.x.

Step 1: Download and install Miniconda

Run the following commands **line by line** in the Terminal.

These commands will:

- create a directory called miniconda3 in your home directory (-p flag stands for parents)
- download the Miniconda installer script for Intel-based Macs
- install Miniconda in silent mode (-b), allow updates (-u), in the given path (-p)
- remove the installer script after installation

```
mkdir -p ~/miniconda3
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-x86_64.sh
-o ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

Step 2: Activate Miniconda

After installation, **close and reopen your Terminal**, or run:

```
source ~/miniconda3/bin/activate
```

You should now see (base) at the beginning of your command prompt.

Step 3: Initialize conda for your shell

Initialize conda so it works automatically in new terminal sessions:

```
conda init
```

Restart your Terminal afterwards if prompted.

Step 4: Verify your installation

Run any conda command. For example: `conda list` (displays a list of packages installed in your active environment and their versions) or `conda --version` (displays conda's version number).

```
(base) name@computer ~ % conda --version
conda 25.7.0
```

Linux

Step 1: Download and install Miniconda

Run the following commands **line by line** to download and install the latest Linux installer for your chosen chip architecture.

These commands will:

- create a directory called `miniconda3` in your home directory.
- download the Linux Miniconda installation script for a 64-bit architecture and save the script as `miniconda.sh` in the `miniconda3` directory. For other chip architectures look up [this link](#).
- run the `miniconda.sh` installation script in silent mode using `bash`.
- remove the `miniconda.sh` installation script file after installation is complete.

```
mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
-o ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

Step 2: Activate Miniconda

After installation, **close and reopen your Terminal**, or run:

```
source ~/miniconda3/bin/activate
```

You should now see `(base)` at the beginning of your command prompt.

Step 3: Initialize conda for your shell

Initialize conda so it works automatically in new terminal sessions:

```
conda init
```

Step 4: Verify your installation

Run any conda command. For example: `conda list` (displays a list of packages installed in your active environment and their versions) or `conda --version` (displays conda's version number).

11.4 4. Understanding Conda Concepts

This section introduces the **core ideas** behind Conda that you need for the rest of the course. The goal is not to memorise terminology, but to understand **how to think about software setups** in a clean and reproducible way.

You will revisit these ideas repeatedly in labs and projects.

11.4.1 What is Conda?

Conda is both a **package manager** and an **environment manager**.

As a package manager, it installs software and libraries that your code depends on. As an environment manager, it keeps different projects separated so they do not interfere with each other.

In practice, Conda helps you answer two questions for every project:

- What software does this project need
- Where should that software live on my machine

This is especially important in geospatial programming, where libraries often depend on additional system software and specific versions need to work together.

11.4.2 What is an environment?

A Conda environment is a **self contained software setup** for a project.

Each environment includes:

- a specific Python version
- a specific set of installed packages
- the system libraries those packages rely on

Environments are isolated from each other. Installing or updating software in one environment does not affect any other environment.

This isolation allows you to:

- work on multiple projects with different requirements
- avoid breaking existing projects when installing new packages
- reproduce the same setup on another computer

Best practice is to create **one environment per project**. If something goes wrong, you can delete the environment and recreate it without affecting your system.

11.4.3 What is a package?

A package (aka. [library](#)) is a **ready to use piece of software** that adds functionality to your environment.

Packages allow you to reuse work done by others instead of writing everything from scratch. They include:

- the software itself
- information about versions and dependencies
- instructions for installation

When you install a package, Conda takes care of:

- downloading the correct files for your system
- installing required dependencies
- making sure everything works together

In geospatial programming, packages often include compiled components in addition to Python code. This is one of the reasons Conda is so useful in this field.

Key takeaway

Think of Conda as a system for **organising software**.

- Conda manages software and environments
- environments define project specific setups
- packages provide reusable functionality

If you understand these three ideas, you understand the foundation of reproducible and reliable geospatial programming.

11.5 5. Working with Conda

This short tutorial walks you through creating and using your first conda environment as part of a real Python workflow. You'll create an environment, install packages, and use that environment to run a small Python program.

11.5.1 Creating an environment

The conda installation process creates an environment called base, which is where conda itself is installed. However, when starting work on a new project, it's best practice to create a new environment. This keeps your environments maintainable and reproducible while also keeping your base environment stable.

Let's create a new environment called sds-env with Python 3.12 as the interpreter.

1. Open a shell application

Conda is a command line interface (CLI) tool, which means you'll use a shell application to run conda commands. For Windows users, you'll use an application called Anaconda Prompt, which comes installed with Anaconda Distribution and Miniconda. For macOS and Linux users, you'll use your system's Terminal application.

Windows

To open Anaconda Prompt, type "Anaconda Prompt" in the Windows search bar, then select Anaconda Prompt.

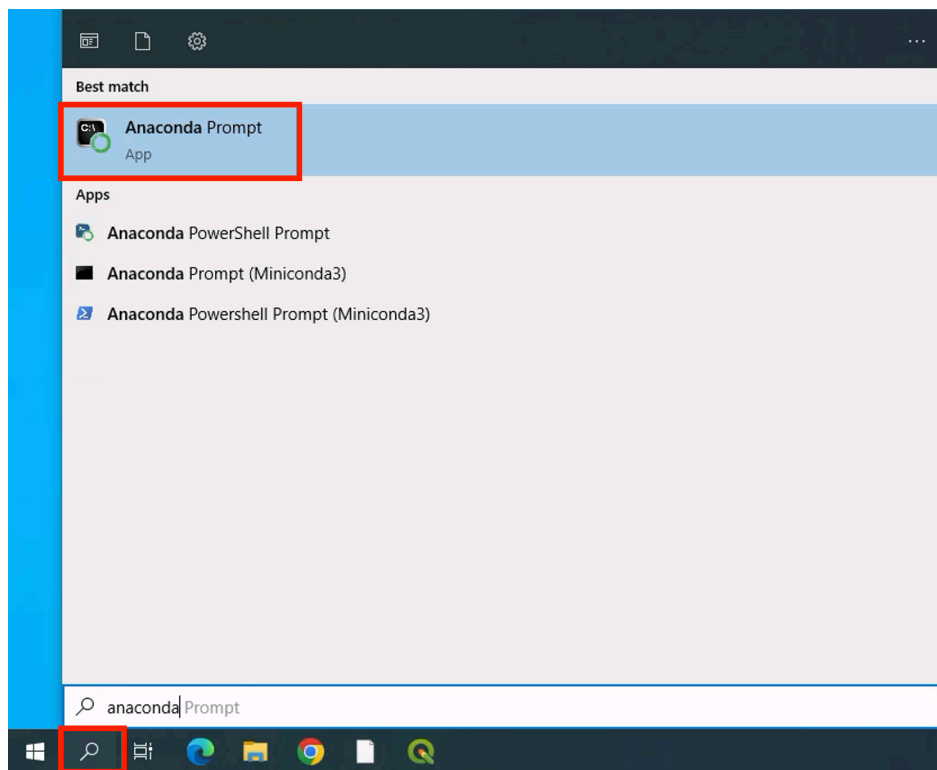


Figure 11.1: Opening the Anaconda Propmt application.

macOS/Linux

Open Terminal:

- On macOS, open Spotlight with Cmd + Spacebar, then search for “Terminal”.
- On Linux, press Ctrl + Alt + T or search for “Terminal” in your application menu.

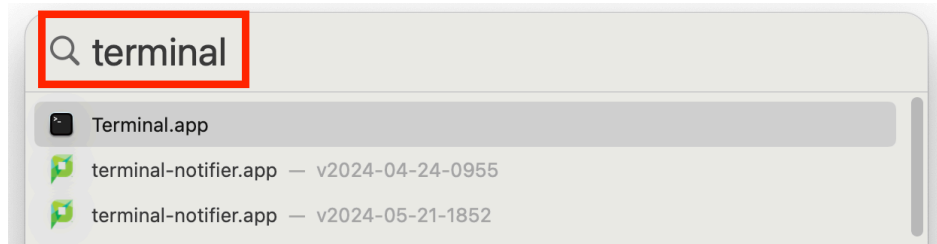


Figure 11.2: Opening the Terminal to use conda.

2. Create a new environment

Use the **copy button**  to copy the command below. Paste it into your shell (Anaconda Prompt, Terminal, or PowerShell) and press Enter (Windows) or Return (macOS/Linux) to run it.

```
conda create --name sds-env python=3.12
```

This command tells Conda to:

- create a **new isolated environment**
- name it `sds-env`
- install **Python 3.12** inside that environment

The `--name` flag assigns a human-readable name, which makes the environment easy to activate and manage later.

When you create a **named** Conda environment, Conda stores it automatically in its default `envs` directory.

Windows

```
C:\Users\<username>\miniconda3\envs\sds-env
```

macOS/Linux

```
/Users/<username>/miniconda3/envs/sds-env
```

You usually do **not** need to interact with this folder directly. Conda takes care of activating, updating, and removing environments for you.

3. Activate your new environment

You'll need to activate your newly created environment before you can use it. Run the following command to activate `sds-env`:

```
conda activate sds-env
```

Conda displays the currently active environment in your shell application beside the input line:

Windows

```
(sds-env) C:\Users\username>
```

```
macOS/Linux
```

```
(sds-env) ~
```

From now on, every Python and conda command you run applies to this environment.

Deactivate your environment when you are done

When you finish working in an environment, you can deactivate it using:

```
conda deactivate
```

After deactivation, you return to the base environment and the environment name disappears from the prompt.

11.5.2 Adding packages to your environment

Right now, your environment only has Python 3.12 and its dependencies installed. However, our project uses functionality that is not provided by the Python standard library, so we must install third-party packages to provide that functionality.

1. Add conda packages

We can find the additional conda packages that we need for our project on the conda-forge community channel.

The input below installs common geospatial packages and their dependencies from the conda-forge channel.

```
conda install --channel conda-forge pygis
```

The `--channel` flag tells conda to give the specified channel top priority for installing packages and their dependencies. The short option `-c` can also be used instead.

2. Add packages with pip

Sometimes you will need a package that is not available through Conda. This is common for smaller or newer Python libraries, including some geospatial helper tools.

In these cases, you can use pip, Python's built in package manager.

When you created your Conda environment, Python was installed automatically, and with it pip. This means you can already use pip inside your active Conda environment.

Assume your project needs a small helper library that is only published on [PyPI](#), for example a lightweight utility for working with map tiles. First, make sure your Conda environment is active. Then install the package using pip:

```
pip install geospatial
```

11.5.3 Working with Conda environments

At this point, you have created an environment and installed packages into it. Now let's look at how to **inspect, update, and manage environments** during day to day work.

Think of this section as learning how to *stay in control* once your environment exists.

1. Checking your environments

It is good practice to start a session by checking which environments exist and which one is currently active.

```
conda info --envs
```

Conda lists all environments on your system. The environment marked with an asterisk is the active one.

2. Inspecting what is installed

To see which packages are installed in an environment, including where they came from, use:

```
conda list --name sds-env --show-channel-urls
```

This helps you understand:

- which packages are installed
- which versions are used
- whether they came from conda-forge or another channel

This is especially useful when debugging or documenting your setup.

3. Installing and removing packages safely

If you need to modify an environment without activating it, you can always target it explicitly.

Install packages into a specific environment:

```
conda install --name sds-env geopandas rasterio
```

Remove a package you no longer need from the environment:

```
conda remove --name sds-env rasterio
```

If a package was installed using pip, it should also be removed using pip.

First, make sure you are targeting the **correct environment** (either by activating it or by using the full path to pip).

Remove a package with pip:

```
pip uninstall geospatial
```

4. Updating packages

Over time, packages receive updates. To update all packages in an environment:

```
conda update --all --name sds-env
```

Use this with care in active projects, as updates may change behaviour. For long term or shared projects, updating should be done intentionally and documented.

5. Copying and cleaning up environments

Sometimes you want to experiment without breaking a working setup.

Clone an existing environment:

```
conda create --clone sds-env --name sds-env-test
```


This creates a full copy that you can safely modify.

When an environment is no longer needed, remove it completely:

```
conda remove --name sds-env-test --all
```

Removing unused environments helps keep your system clean and avoids confusion later.

6. Exporting environments for reproducibility

One of the most important steps in professional workflows is **documenting your environment**.

Export an environment to a file (to your current working directory):

```
conda env export --name sds-env > sds-env.yml
```

This file records package names and versions so others can recreate the same setup.

Recreate an environment from such a file:

```
conda env create --name sds-env-copy --file sds-env.yml
```

11.5.4 A typical Conda workflow

By now, you have seen the full lifecycle of working with Conda for a project. The exact package names will change, but the workflow stays the same.

```
# create a new environment for the project
conda create -n sds-env python=3.12

# activate the environment
conda activate sds-env

# install required geospatial packages
conda install -c conda-forge geopandas rasterio

# check what is installed
conda list

# work on your project
python analysis.py

# export the environment for reproducibility
conda env export > environment.yml

# deactivate when you are done
conda deactivate
```

What matters is not the exact commands, but the pattern:

- one environment per project
- activate before you work
- prefer conda-forge for geospatial packages
- use pip only when Conda is not an option
- document environments for reproducibility

You will repeat this workflow throughout your geospatial projects. Over time, it becomes second nature.

11.6 6. Working with uv

So far, we introduced **Conda**, which is the main tool used in this course for managing geospatial environments. Conda is reliable and well suited for packages with complex system dependencies.

Sometimes, however, you may want a **lighter and faster tool** for Python only workflows. This is where **uv** comes in.

11.6.1 What is uv and when to use it?

uv is a modern Python package manager written in [Rust](#). It is designed to be *extremely fast* while remaining compatible with Python's existing ecosystem.

Use **Conda** when:

- working with geospatial libraries that depend on system software
- installing packages like GDAL, GEOS, PROJ, geopandas, or rasterio
- reproducibility across operating systems matters

Use **uv** when:

- working with pure Python packages from [PyPI](#)
- setting up small or temporary environments
- prototyping quickly or running experiments

In this course, **Conda remains the default**, but uv is a useful complement to know.

11.6.2 Installing and creating an environment

You only need to install uv once to your base environment.

Windows

```
irm https://astral.sh/uv/install.ps1 | iex
```

macOS/Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Navigate to your project directory and create a virtual environment:

```
cd path/to/your/project
uv venv --python 3.12
```

Activate the environment:

Windows

```
.venv\Scripts\activate
```

macOS/Linux

```
source .venv/bin/activate
```

11.6.3 Installing and running packages

Install packages at high speed:

```
uv pip install jupyterlab leafmap
```

Install from a requirements file:

```
uv pip install -r requirements.txt
```

Run Python or tools directly inside the environment:

```
uv run python script.py
```

```
uv run jupyter lab
```

11.6.4 A typical uv workflow

With uv, environments usually live **inside the project folder**. This makes each lab self-contained and easy to reset.

Assume you are working in your **SDS-labs directory**:

```
SDS-labs/  
└─ lab-03/
```

11.6.4.1 1. Go to your lab directory

```
cd SDS-labs/lab-03
```

This is where the environment will live.

11.6.4.2 2. Create a virtual environment

```
uv venv
```

This creates a local environment:

```
lab-03/  
├─ .venv/          ← virtual environment  
└─ notebooks/
```

You usually do **not** touch `.venv` directly.

11.6.4.3 3. Activate the environment

```
source .venv/bin/activate      # macOS / Linux  
.venv\Scripts\activate         # Windows
```

Your shell now uses the Python from `.venv`.

11.6.4.4 4. Install packages for the lab

```
uv pip install geopandas rasterio jupyterlab
```

Packages are installed **only for this lab**, not globally.

To check what is installed:

```
uv pip list
```

11.6.4.5 5. Start JupyterLab

```
jupyter lab
```

- Jupyter opens **in your browser**
 - The notebook kernel uses the `.venv` environment
 - You work entirely in the browser
-

11.6.4.6 6. Finish your work

1. **Stop JupyterLab** In the terminal:

Ctrl + C

1. **Deactivate the environment**

`deactivate`

Your system is now clean again.

11.6.4.7 7. (Optional) Record dependencies

To keep track of what was installed:

```
uv pip freeze > requirements.txt
```

This file allows you (or someone else) to recreate the environment later.

11.6.5 Conda vs uv at a glance

Both workflows follow the same logic:

create → activate → install → work → stop → deactivate

11.6.6 Key takeaways

- Conda is your main tool for geospatial environments
- uv is a fast option for Python only tasks
- Knowing both helps you choose the right tool for the job

This is not about using more tools, but about **using the right tool in the right context**.

11.7 7. Best practices

As your projects grow, good habits around environments will save you a lot of time and frustration. The goal is not perfection, but **clarity and control**.

1. Keep environments simple and focused

Each project should have its **own environment**.

Good practice:

- use descriptive but short names such as `sds-env`, `sds210-lab1`, or `thesis-env`
- install only what the project actually needs
- remove environments you no longer use

Avoid putting everything into one large environment. Large environments are harder to debug, slower to solve, and difficult to reproduce.

2. Document environments early

An environment is part of your project, just like your code.

- export environments to a file
- keep the file with your project
- update it when dependencies change

This makes your work reproducible for:

- yourself in the future
- your lab partner
- your supervisor

- automated systems

You do not need to export after every small change, but do it whenever the environment becomes important.

3. Prefer reliable packages sources

Installing packages is easy. Installing them **well** takes a bit of discipline. For geospatial work:

- prefer conda-forge
- specify the channel explicitly
- install Conda packages before using pip

This reduces conflicts and improves reproducibility.

4. Update and troubleshoot intentionally

When things go wrong:

- check which environment is active
- read error messages carefully
- update conda or mamba before trying random fixes

If installations behave strangely, clearing cached packages can help:

```
conda clean --all
```

Big picture

You do not need to [memorise commands](#), but you should internalise these ideas:

- environments isolate projects and prevent conflicts
- conda-forge is essential for geospatial work
- environments are part of your project documentation
- clean setups lead to reliable results

Avoid installing packages into:

- the system Python
- the Conda base environment

Instead, create a new environment for each project. This habit alone will prevent many hard-to-debug problems later on.

By following these practices, you are not just learning tools. You are learning **professional workflows** that scale from coursework to research and real-world projects.

11.8 8. Exercises

These exercises help you practice the core skills needed to work confidently with Conda in real projects.

11.8.1 Exercise 1: Creating and Using a Project Environment

Goal: Create a clean Conda environment and use it for geospatial work.

1. Create a new Conda environment called `sds-env` with Python 3.12
2. Activate the environment
3. Install the `pygis` package from the conda-forge channel
4. List all installed packages in the environment

5. Start Python and verify that the environment is working
 6. Install `geopandas` and `rasterio` explicitly into `sds-env`
 7. Check which channel each package was installed from
 8. Deactivate the environment and confirm that it is no longer active
 9. Create a second environment called `sds-env-alt` with Python 3.12
 10. Compare the installed packages between the two environments
-

11.8.2 Exercise 2: Inspecting and Managing Environments

Goal: Learn how to stay in control once environments exist.

1. List all Conda environments on your system
 2. Identify which environment is currently active
 3. Inspect the installed packages in `sds-env`, including their source channels
 4. Install an additional package into `sds-env` without activating it
 5. Remove that package again
 6. Update all packages in `sds-env`
 7. Clone `sds-env` into a new environment called `sds-env-test`
 8. Remove `sds-env-test` once you are done
-

11.8.3 Exercise 3: Reproducibility with Environment Files

Goal: Practice exporting and recreating environments.

1. Export the `sds-env` environment to a file called `sds-env.yml`
 2. Create a new environment called `sds-env-copy` from that file
 3. Activate `sds-env-copy` and verify that it works
 4. Compare the package lists of `sds-env` and `sds-env-copy`
 5. Add one additional package to `sds-env-copy`
 6. Export the updated environment to a new file
 7. Remove the original `sds-env`
 8. Recreate it only from the exported file
-

Reflection

After completing the exercises, take a moment to reflect:

- Which step felt most confusing
- Which command felt most useful
- What would you do differently next time

Being comfortable with environments is less about memorising commands and more about **knowing what to check when something breaks**.

If you can create, inspect, export, and recreate environments confidently, you have mastered one of the most important foundations of professional geospatial programming.

12. VS Code

Setting Up Your Code Development Environment

12.1 1. Introduction

Visual Studio Code (VS Code) is a lightweight, extensible code editor. It is a place where you can write code, run it, debug errors, explore data, and keep your work organised.

Think of VS Code as your **digital workspace**. Instead of switching between many tools, it brings code editing, terminals, notebooks, and version control together in one place. VS Code understands Python and helps you write cleaner code and spot problems earlier.

VS Code is especially well suited for **geospatial programming**. You can work with Python scripts and Jupyter notebooks side by side, run Conda commands in an integrated terminal, and track changes with Git without leaving the editor. As projects grow in size and complexity, this integration becomes increasingly important.

Another reason we promote VS Code is **balance**. It is lightweight enough to start quickly, but powerful enough to support professional workflows. You can begin with a minimal setup and gradually add features as you need them.

In this section, you will learn how VS Code fits into the python workflow and how to use it as a **reliable development environment** for geospatial programming.

12.2 2. Learning Objectives

After working through this section, you should be able to:

- **Work productively in VS Code as your main workspace**
- **Select and verify the correct Conda interpreter in VS Code**
- **Run Python scripts and Jupyter notebooks inside VS Code**

These objectives focus on using VS Code as part of a practical workflow. You will build on them throughout the course as your projects become more complex.

12.3 3. Installing VS Code

To install Visual Studio Code go to the official website: <https://code.visualstudio.com>

The site automatically detects your operating system and suggests the correct installer.

VS Code is available for **Windows**, **macOS**, and **Linux**, and the installation is quick on all platforms.

Windows

- Download the Windows installer (.exe)
- Run the installer and follow the setup wizard
- Keep the default options unless you know you want something different

macOS

- Download the macOS installer (.dmg)
- Open the file and drag **Visual Studio Code app** into the Applications folder
- Start VS Code from Applications or Spotlight
- On macOS, you may need to confirm that you trust the application the first time you open it.
- Optional but recommended: keep VS Code in your Dock for quick access

Linux

- Download the appropriate package for your distribution (.deb or .rpm)
- Install it using your package manager or by opening the downloaded file
- Alternatively, follow the instructions on the [VS Code website](#) for your distribution

When you open VS Code for the first time, you will see a welcome screen with helpful getting-started information.

To help you navigate the different features and panels more easily, you may want to watch this beginner tutorial.

At this point, VS Code is installed and ready. In the next sections, you will set it up for Python and geospatial work.

12.4 4. Essential Extensions

VS Code becomes powerful through **extensions**. Extensions add language support and tools that turn the editor into a full development environment. For this course, you only need a **small core set** to get started.

You can always add more later. Start simple.

12.4.1 Core extensions

These two extensions are essential for this course.

Python by Microsoft

This is the foundation for all Python work in VS Code.

It provides:

- Code completion and syntax highlighting
- Error detection and basic linting
- Debugging support
- Integration with Conda environments

Without this extension, VS Code does not really understand Python.

Jupyter by Microsoft

This extension adds native support for Jupyter notebooks inside VS Code.

It allows you to:

- Open and run notebooks (.ipynb) directly in the editor
- Select Conda environments as notebook kernels
- Mix code, text, and output in one place

This is especially useful for exploratory geospatial analysis.

12.4.2 Installing extensions

There are two common ways to install extensions.

Using the Extensions panel

1. Click the Extensions icon in the left sidebar
2. Search for the extension name
3. Click Install

Using the Command Palette

1. Open the Command Palette
 - Windows and Linux: Ctrl + Shift + P
 - macOS: Cmd + Shift + P
2. Type Extensions: Install Extensions
3. Search and install the extension

Once installed, extensions activate automatically.

If you get stuck, you might find it helpful to watch this video on VS Code extensions.

12.4.3 What not to install (yet)

VS Code offers thousands of extensions. It is tempting to install many of them, but this often causes confusion early on.

For now:

- Do not install formatters, linters, or AI tools (unless instructed)
- Do not change many settings

You will add more extensions later, once you understand your workflow better.

In the next subsection, you will connect VS Code to your Conda environment and run your first Python code inside the editor.

12.5 5. Connecting VS Code to Conda

VS Code does not automatically know which Python environment you want to use. To make sure your code runs with the **right packages and the right Python version**, you must explicitly connect VS Code to your Conda environment.

In this course, this step is essential. Most setup problems later come from using the **wrong environment**.

12.5.1 Why this matters

When VS Code is connected to a Conda environment, it will:

- run Python code using the correct interpreter
- find the packages you installed with Conda
- use the same environment in scripts, notebooks, and the terminal
- avoid confusing situations where code works in one place but not another

Think of this as telling VS Code *this is the toolbox I want you to use for this project*.

12.5.2 Selecting a Conda environment

Before selecting an environment, always open your **project folder** in VS Code.

- Go to **File** → **Open Folder**
- Select the folder of your project
- This folder becomes your workspace

VS Code remembers the Python environment **per workspace**, not globally.

Then select the environment:

Use the Command Palette to select the correct interpreter.

- Open the Command Palette

- Windows or Linux: Ctrl + Shift + P
 - macOS: Cmd + Shift + P
- Type **Python: Select Interpreter**
- Choose your Conda environment for example sds-env (conda)

Once selected, VS Code stores this choice for the current workspace.

12.5.3 Verifying and using the environment

You can always check whether VS Code is using the correct environment.

- Look at the **status bar** at the bottom of VS Code You should see the environment name and Python version
- Or run this in a Python file:

```
import sys
print(sys.executable)
```

The printed path should point to your Conda environment, not to a system Python.

For Jupyter notebooks:

- Open a .ipynb file
- Click **Select Kernel** in the top right
- Choose the same Conda environment you selected before

This keeps notebooks consistent with your scripts.

Once VS Code is connected to Conda, you are ready to work productively writing code, running notebooks, and managing projects with confidence.

12.6 6. Working with VS Code

Now that VS Code is installed and connected to your Conda environment, this section focuses on **how you actually work in it day to day**. The goal is not to learn every feature, but to become comfortable with a small set of workflows that you will use throughout the course.

“From here on, VS Code is a convinient place where you write code, run notebooks, and manage your projects.”

Before diving into Python, notebooks, or geospatial libraries, it is worth getting comfortable with the basic ideas behind how VS Code works. This will save you time and reduce confusion later.

12.6.1 The main interface

VS Code looks simple at first, but it is built around a few clearly defined areas. The labels in the figure correspond to the sections below.

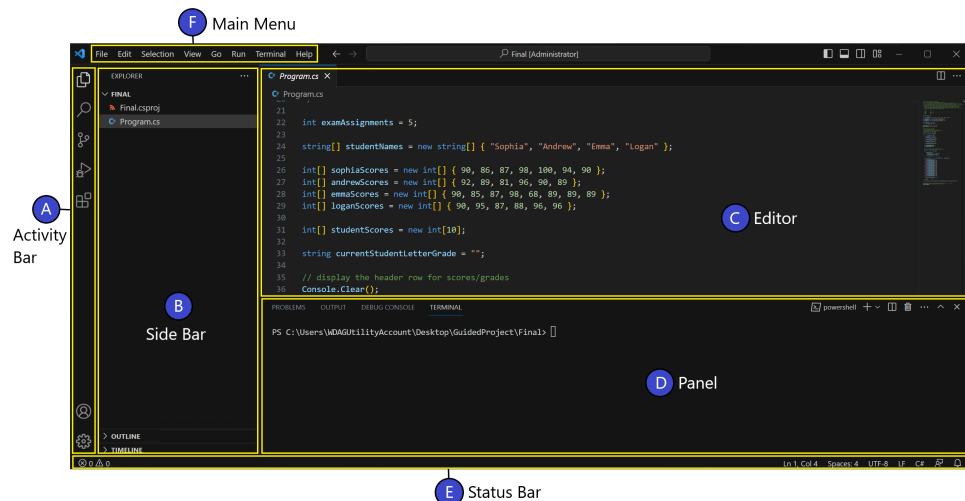


Figure 12.1: Visual Studio Code user interface.

A. Activity Bar The Activity Bar runs vertically on the far left. It lets you switch between major views such as the **Explorer**, Search, Source Control (Git), Run, and Extensions. Think of it as the navigation spine of VS Code.

B. Side Bar The Side Bar shows the content of the currently selected activity. Most of the time, this is the **Explorer**, where you see your project folders and files. When you switch activities, the Side Bar adapts accordingly.

C. Editor The Editor is the main working area in the center. This is where you write Python scripts, edit notebooks, view files, and inspect results. You can open multiple files side by side and switch between them using tabs.

D. Panel The Panel appears at the bottom and hosts tools such as the **Terminal**, Output, Debug Console, and Problems view. You will frequently use the **Terminal** here to activate Conda environments, install packages, and run code without leaving VS Code.

E. Status Bar The Status Bar runs along the bottom edge of the window. It shows important context such as the active Python interpreter, Conda environment, file type, and cursor position. If something behaves unexpectedly, this is often the first place to check.

F. Main Menu The Main Menu at the top provides access to all VS Code functionality, including opening folders, managing settings, running code, and controlling the editor layout. Many menu actions are also available through the Command Palette.

You do not need to memorize all of this. As you work, these areas will quickly become familiar and help you stay oriented while coding and exploring data. You will likely spend most of your time in the Editor, Explorer, Terminal, and Status Bar.

12.6.2 The workspace

VS Code works around the idea of a **workspace**, which is simply a folder on your computer.

When you open a folder in VS Code:

- all files in that folder become part of your project
- settings such as the selected Python environment are stored for that project
- the integrated terminal opens in the correct directory

See your **project folder** as a place that keeps your code, data, notebooks, and environment configuration in one place.

12.6.3 The Command Palette

The most powerful feature in VS Code is the **Command Palette**.

Open it with:

- Ctrl + Shift + P on Windows and Linux
- Cmd + Shift + P on macOS

The Command Palette lets you:

- select Python interpreters
- install extensions
- open settings
- run VS Code features without menus

If you are unsure where something is in VS Code, open the Command Palette and search for it.

12.6.4 Integrated terminal

VS Code includes a built-in terminal that runs inside your workspace.

You can use it to:

- activate Conda environments
- install packages
- run Python scripts
- start Jupyter notebooks

Open it via: View arrow.r Terminal

The terminal automatically starts in your project folder, which avoids many path related mistakes.

12.6.5 Editing and running code

VS Code is more than a text editor. It understands Python and helps you write better code.

Typical workflow:

1. Create a new Python file
2. Write code with syntax highlighting and auto completion
3. Run the file directly from VS Code
4. Inspect output in the integrated terminal

You can run a Python file in several ways:

- Run arrow.r Run Without Debugging
- Right click in the editor arrow.r Run Python File
- Use the integrated terminal

VS Code automatically uses the Python interpreter from your selected Conda environment.

12.6.6 Jupyter notebooks

VS Code also supports **Jupyter notebooks** directly inside the editor.

This allows you to:

- explore data interactively
- mix code, output, and explanations
- use the same Conda environment as your scripts

Scripts and notebooks can live side by side in the same project and use the same environment.

In this course, you will mainly use the following files:

- .ipynb files for Jupyter notebooks
- .py files for Python scripts
- .yaml files for Conda environments
- folders such as data, notebooks, and src (short for source)

VS Code understands all of these file types and adapts its interface automatically.

VS Code is more than a text editor. It keeps code, environments, and notebooks connected at the project level. Once you open the right folder and select the right environment, most things just work.

12.7 7. Exercises

These exercises help you practise the **basic VS Code workflows** you can use throughout SDS210. They are designed to be short, practical, and focused on habits rather than features. Take your time. If something does not work, use it as a chance to explore the interface and ask questions.

12.7.1 Exercise 1: Opening a project workspace

Goal Learn how VS Code works with folders and projects.

1. Create a new folder on your computer called sds210-vscode
2. Open VS Code
3. Use **File arrow.r Open Folder** and select sds210-vscode
4. Inside VS Code, create the following folder structure:
 - data
 - notebooks
 - src
5. Create a file src/DIDit.py with:

```
print("Every machine that is clever enough to pass the Turing test could also  
be clever enough not to pass it.")
```

1. Save the file

This folder is now your **workspace**. VS Code will remember settings for this project.

12.7.2 Exercise 2: Connecting VS Code to Conda

Goal Make sure VS Code uses the correct Python environment.

1. Open the **Command Palette**
 - Windows or Linux: Ctrl + Shift + P
 - macOS: Cmd + Shift + P
2. Select **Python: Select Interpreter**
3. Choose your Conda environment (for example sds-env)
4. Open DIDit.py
5. Run the file using **Run arrow.r Run Without Debugging**

Check:

- Output appears in the integrated terminal
- The **Status Bar** shows the selected Conda environment

Optional check:

```
import sys
print(sys.executable)
```

The path should point to your Conda environment.

12.7.3 Exercise 3: Working with terminal and notebooks

Goal Use VS Code as a single place for code, commands, and notebooks.

1. Open the **integrated terminal**
2. Activate your Conda environment
3. Run:

```
python --version
```

1. Create a new notebook notebooks/DUTgemacht.ipynb
2. Select the same Conda environment as the kernel
3. Run a code cell with:

```
print("Die Proletarier haben nichts zu verlieren als ihre Goldkettchen.")
```

Confirm that:

- The notebook runs without errors
 - Scripts and notebooks use the same environment
-

13. Git

Keeping track of change

13.1 1. Introduction

At some point, this might happen: you delete a function that *definitely* worked yesterday, a notebook experiment goes wrong, or you realise that your `final_final_v3.ipynb` is not final at all.

This is where **Git** comes in.

Git is a version control system that **tracks every change** you make to your project files. It builds a complete history of your project, step by step. You can go back in time, compare versions, undo mistakes, and understand *what changed, when, and why*. Think of Git as a **never-forget undo button** for your code and notebooks.

This is especially useful in spatial data science, where projects evolve through trial and error. With Git, you can:

- experiment freely without losing any work
- understand what changed and why
- collaborate without overwriting each other

Git works locally on your computer and integrates well with tools you will use, such as VS Code and Jupyter notebooks. Combined with platforms like **GitHub**, it also lets you back up and share your work.

13.2 2. Learning Objectives

After working through this section, you should be able to:

- use **Git** to track changes in code and notebooks
- save and restore project states using commits
- sync a local project with **GitHub**

These objectives focus on using Git as a **practical safety net** in your daily workflow. You will keep building on them as your projects grow and become more collaborative.

13.3 3. Git vs. GitHub

Before learning any commands, it's important to get the **big picture** right. One of the most common sources of confusion is mixing up **Git** and **GitHub**.

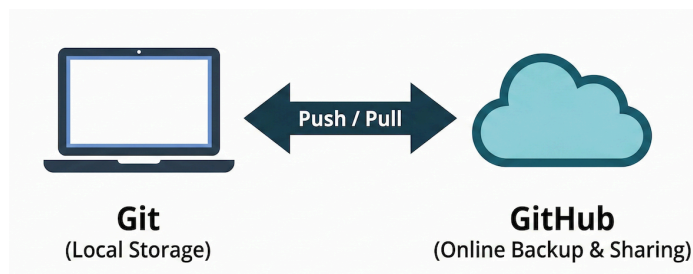


Figure 13.1: Git is the software running on your computer. GitHub is the online platform where you store and share your Git repositories.

Git runs on *your computer*. It tracks changes to your files and stores the full history of your project locally. You can use Git completely offline. A Git repository is best thought of as a **timeline** of your project. Each point on that timeline is a **commit**, a snapshot of the entire project at a specific moment. Every commit includes all tracked files exactly as they looked then, plus a **short message** explaining what changed.

How Git thinks: Git does **not** save “versions of files” (like `script_v1`, `script_v2`). It saves **project states over time**. If a file did not change, Git simply reuses it internally. Because every commit is a snapshot, nothing is ever overwritten. The past is still there, so you can always go back or undo mistakes without panic.

GitHub lives *online*. It hosts Git repositories on the web and acts as a **shared, central copy** of your project. GitHub allows you to back up your work safely, collaborate with others without overwriting their changes, and share your work publicly. GitHub does not track changes by itself; it **stores and synchronises** the history created by Git.

What happens where?

- **Locally (Git):** Edit files, track changes, create commits, view history.
 - **Online (GitHub):** Store a backup, sync between machines, collaborate, share code.
-

13.4 4. Git Setup

Before using Git in practice, we set it up **once**. This removes friction later and avoids confusing errors.

Step 1: GitHub account Log in to your [GitHub account](#). If you don't have one yet, [create an account](#). Use an email address you'll remember, as you'll need the same one for Git.

Step 2: Git installation Check in your Terminal whether **Git** is already installed:

```
# Check Git version
git --version

# Check installation location
which git    # macOS/Linux
where git    # Windows
```

If Git responds with a version number, you're good. If not, use [this link](#) to install Git for your operating system, test it with `git --version`, and come back here.

Step 3: Identity configuration Git needs to know who you are. Set your name and email **once**:

```
# Replace with your actual name and GitHub email
git config --global user.name "Your Name"
git config --global user.email "your@email"
```

This information is attached to every commit you make.

Step 4: Verify everything Check your setup:

```
# View all global configuration
git config --global --list
```

13.5 5. Understanding Git

Before diving into commands, let's understand the key concepts that make Git powerful for geospatial programming.

13.5.1 Core Git Concepts

- **Repository (Repo):** A project folder that Git watches. It contains your files *and* the hidden information Git uses to track changes and history.
- **Commit:** A snapshot of the project at a specific moment in time. Think of it as a reliable **save point**. Each commit has a unique ID, includes a short message explaining what changed, and once created, it is a state you can always return to.
- **Branch:** A parallel timeline of the same project. The main branch contains stable, working code, while other branches are used for experiments. They let you try things out without breaking what already works.

- **Remote:** A linked copy of your repository stored online (e.g., on GitHub). While Git manages history locally, remotes allow you to back up your work, sync between machines, and collaborate.

13.5.2 The Git Workflow

Almost everything you do with **Git** follows the **same simple loop**:

Edit → Stage → Commit → Push

If you understand this loop, Git will feel familiar.

1. **Working Directory (*edit*):** This is your normal project folder. You edit files here (scripts, notebooks). At this point, Git is only *watching*, and nothing is saved yet.
2. **Staging Area (*select*):** This is where you **choose** which changes should go into the next snapshot. You decided what changes form a logical step together; you don't have to commit everything you edited at once.

Think of staging as saying: *"These specific changes are ready to be saved."*

3. **Commit (*save a snapshot*):** This creates a permanent **snapshot of the staged changes** and adds it to the project history. Once committed, that state is safe.
4. **Push / Pull (*sync with GitHub*):** These commands align the histories of your computer and your linked GitHub remote.

- **push** arrow.r send your local commits to **GitHub** (to back up or share).
- **pull** arrow.r bring remote changes from GitHub back to your computer.

13.5.3 Git in VS Code

VS Code includes built-in Git support. If Git is installed on your system, no additional extensions are required.

In the **Source Control** panel (the branch icon on the left), you can see the same workflow you just learned visualized: modified files move to staging (+), staged files move to commit, and committed changes move to push. VS Code also provides a visual diff viewer, which helps you inspect changes line by line before staging them.

13.5.4 Key Takeaway

Working directory → Staging area → Local history → GitHub
(edit) (add) (commit) (push)



Figure 13.2: The core Git loop: editing files, staging the specific changes you want to save, committing them to history, and pushing the backup online.

Git is not a collection of random commands. It's one loop, repeated over and over — and it will be the backbone of this course.

13.6 6. Working with Git

In this section, you will learn how to use Git in practice. We will cover two distinct workflows you will use in this course: syncing the official course material, and managing your own personal projects.

13.6.1 Workflow A: Syncing the SDS210 Course Material

Throughout this course, new practical notebooks are released weekly on GitLab. Instead of downloading ZIP files every week, you can use Git to download the repository **once** and update it with a single command.

1. Navigate to your desired location Open your terminal (*Terminal* on macOS/Linux, *Git Bash* or *Anaconda Prompt* on Windows). Decide where you want to store the course folder and use `cd` (change directory) to go there.

```
# Example (macOS/Linux):  
cd ~/Documents
```

```
# Example (Windows, Git Bash):  
cd /c/Users/<username>/Documents
```

(Tip: Type `ls` to list the files in your current folder to ensure you are in the right place.)

2. Clone the repository (Do this ONCE) Download the repository to your computer. This creates a new folder called `sds210/` in your current directory.

```
git clone https://gitlab.com/HendrikWulf/sds210.git
```

3. Enter the repository folder This is the step that is most frequently forgotten! To run Git commands on the course material, you must be **inside** the folder that contains the hidden `.git` directory.

```
cd sds210
```

If you list (`ls`) all files (`-a`) you should see the course files and folders, including the hidden `.git` folder:

```
ls -a
```

4. Update the repository (Do this WEEKLY) Whenever new notebooks are added to the course, open your terminal, navigate inside your `sds210` folder, and pull the updates:

```
git pull
```

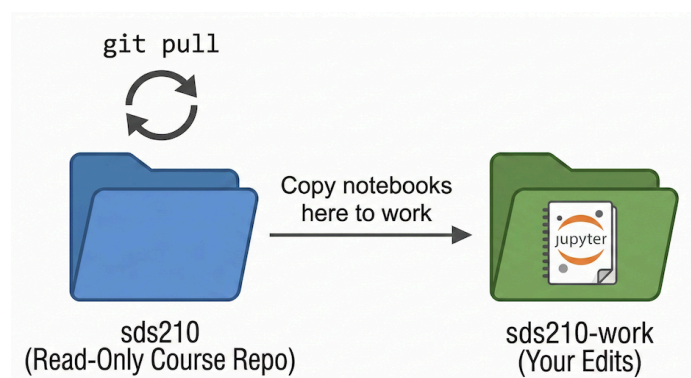


Figure 13.3: Always copy notebooks out of the main course repository into your personal work folder before editing them to prevent merge conflicts.

13.6.1.1 Troubleshooting Course Updates

- **“fatal: not a git repository”** You are trying to pull, but you are not inside the cloned folder. *Fix:* Run `cd path/to/sds210` first.
- **“Updates were rejected” / Merge conflicts** This happens if you accidentally modified files directly inside the repo instead of your work folder. *Fix:* Move your

modified notebook to a safe location outside the folder or rename them. Then, you can force the repo to match the online version using `git restore .` followed by `git pull`.

13.6.2 Workflow B: Starting Your Own Project

When you are starting **your own personal project** from scratch (like your final assignment), you won't clone an existing repository. Instead, you will create an empty folder and turn it into a Git repository.

```
cd ~/Documents/my-geo-project
git init      # Turn the empty folder into a Git repository
git status    # Check the repository status
```

Everything now lives **only on your computer** until you link it to GitHub.

13.6.2.1 Repository Structure & Ignoring Files

A clear repository structure makes Git more useful. For your personal projects, a simple structure is enough:

```
my-project/
├── README.md
├── .gitignore
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
└── outputs/
```

13.6.2.2 Tracking Your Changes

Once your personal repository is set up, the everyday workflow starts: **edit, stage, commit**.

```
# 1. Check the current state
git status  # Tells you WHAT is different
git diff    # Tells you exactly HOW files changed line-by-line

# 2. Stage changes (Select what belongs in this snapshot)
git add analysis.py
git add .      # Alternatively, stage ALL changes in the folder

# 3. Commit changes (Save the snapshot)
git commit -m "Add NDVI calculation for Landsat 8 imagery"

# Shortcut: Stage and commit all already-tracked files at once
git commit -am "Update visualization parameters"
```

*Note: Commit messages should be short (under 50 chars), start with a verb (Add, Fix, Update), and clearly describe **what changed and why**.*

13.6.2.3 Backing Up to GitHub

To safely back up your personal project and share it, you need to link your local repository to GitHub.

```
# Connect local repo to GitHub (Run ONCE per project)
git remote add origin [https://github.com/](https://github.com/)<your-username>.<repo-name>.git

# Verify the connection worked
git remote -v

# Push changes to GitHub for the FIRST time (sets default branch)
git push -u origin main

# All subsequent pushes
git push
```

```
# If collaborating, download others' updates to your computer
git pull
```

13.6.3 Basic Branching

Branching lets you work on **new ideas without breaking what already works**. Instead of changing your main project directly, you create a **separate timeline** for experiments or fixes.

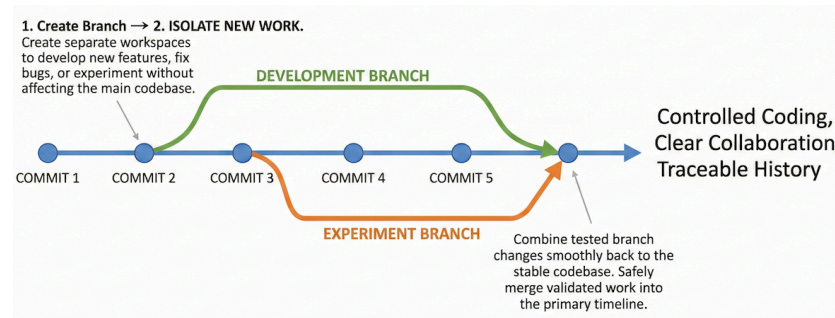


Figure 13.4: Branching allows you to safely experiment on a parallel timeline without affecting the stable main branch until you are ready to merge.

```
# Create a new branch called 'experiment' and switch to it immediately
git checkout -b experiment

# Check which branch you are currently on
git branch

# ... edit files, add, and commit your experimental work ...

# Switch back to the stable main branch
git checkout main

# Merge your successful experiment into the main branch
git merge experiment

# Delete the branch after a successful merge to keep things tidy
git branch -d experiment
```

13.6.4 Viewing History and Undoing

One of Git's biggest strengths is that it keeps a complete history of your project.

```
# Show the full commit history (press 'q' to exit)
git log

# Show a compact one-line history (often the most useful)
git log --oneline

# Show commits that affected one specific file
git log -- data_processing.py
```

Sometimes you realise that the **last commit was a mistake** (e.g., the message was wrong or it was too early). To undo the last commit **without losing your work**:

```
# Undo the last commit BUT keep your file changes safe in the staging area
git reset --soft HEAD~1
```

(Note: Only use `git reset` on commits that **have not been pushed to GitHub yet**.)

13.7 7. Exercises

These exercises focus on using Git as a **practical safety net**. Use the section above as a reference.

13.7.0.1 Exercise 1: Git setup check

Objective: Confirm that Git is correctly installed and configured.

1. Open a terminal.
2. Check that Git is installed:

```
git --version
```

1. Check your Git identity:

```
git config --global user.name  
git config --global user.email
```

1. Verify that your name and email are set correctly and match your GitHub account.

Expected outcome:

- Git responds with a version number.
- Your name and email are correctly configured.
- You are ready to use Git with GitHub.

If this exercise fails, **stop here and fix it** before continuing.

13.7.0.2 Exercise 2: Your project repository

Objective: Create a clean project repository and track its first changes.

1. Create a new folder for a project (e.g. my-first-git-project).
2. Initialise a Git repository inside it:

```
git init
```

1. Create the following structure:

```
my-first-git-project/  
├── README.md  
├── data/  
├── notebooks/  
└── src/
```

1. Write a short description of the project in README.md.
2. Check the repository status:

```
git status
```

1. Stage and commit your changes with a meaningful message.
2. Create a new repository on GitHub and connect it as a remote.
3. Push your commit to GitHub.

Expected outcome:

- A GitHub repository with:
 - a clear folder structure
 - a readable README
 - at least one commit
 - You understand the loop: *edit arrow.r stage arrow.r commit arrow.r push*.
-

13.7.0.3 Exercise 3: Safe experimentation with branches

Objective: Practice experimenting safely using Git history.

1. Create a new branch called experiment:

```
git checkout -b experiment
```

1. Make a small change (e.g. edit the README or add a comment to a file).
2. Commit the change.
3. Switch back to the main branch.
4. Inspect the commit history:

```
git log --oneline
```

1. Merge the experiment branch into main.
2. Delete the branch after merging.

Expected outcome:

- You understand that branches are **safe workspaces**.
- You can inspect project history and see how changes evolve.
- You feel confident experimenting without fear of breaking things.

After these exercises, you should feel that Git helps you **stay in control**, mistakes are **recoverable**, and Git is a tool that *supports* your learning rather than getting in the way.

14. Colab

Running Jupyter notebooks in the cloud

14.1 1. Introduction

Google [Colab](#) is a **cloud based Jupyter notebook** that runs entirely in your browser. Your code executes on remote machines, not on your laptop, which means you can start working **immediately** without installing anything.

In this course, Colab is a **low barrier entry point** to programming with spatial data. It allows you to focus on learning Python, understanding geospatial workflows, and experimenting with code, rather than spending time on software setup and configuration. When your local environment is not ready or something breaks, Colab is your reliable fallback.

Colab is also a **shared learning space**. Notebooks can be opened, run, and edited by anyone with a link. This makes it well suited for live demonstrations, guided exercises, peer collaboration, and asking for help.

Throughout this course, you can use Colab to **run all course notebooks**, modify code examples, and explore spatial data workflows in a clean, consistent, and reproducible environment.

14.2 2. Learning Objectives

After working through this section, you should be able to:

- Open and run **course notebooks** in Colab
- Install and use required **Python packages**
- Manage **files and data** in Colab using Google Drive

These objectives focus on using Colab as a practical learning and experimentation environment. You will use these skills repeatedly when working through examples and practicals in the course.

14.3 3. Getting Started

Getting started with Colab is simple and only takes a few minutes.

First, open [Colab](#). You only need a **web browser** and a **Google account**.

Once you are signed in, create a new notebook by clicking (File -> New notebook in Drive). This opens an empty Jupyter notebook on your Google Drive where you can start writing and running Python code immediately.

Each notebook consists of **code cells** and **text cells**. Code cells are used to run Python. Text cells are used to explain what your code does. You run a cell with **Shift Enter**.

Colab saves your notebook automatically, but the running session is temporary. This means you can always reopen the notebook later, but you may need to rerun cells when you come back to it.

For now, the key idea is simple. Open Colab, create a notebook, write code in cells, and run them from top to bottom.

14.4 4. The Notebook Environment

A Colab notebook is made up of **cells** that are executed one after another. Each cell can contain either Python code or explanatory text. The order in which you run cells matters, because variables and results are stored in memory.

Colab runs your code in a **session**. This session holds all variables, imported libraries, and loaded data. If the session stops or resets, everything in memory is lost and cells need to be run again.

The **execution order** is not always the same as the visual order of cells. If you run cells out of order, results may be confusing or incorrect. When something behaves strangely, a good habit is to restart the session and run all cells from top to bottom.

The **runtime** defines where your code runs and which resources are available. Most of the time, the default runtime is sufficient (Python 3 on a CPU). In later project, you may switch runtimes for more demanding tasks (Runtime -> Change Runtime type, GPU or TPU).

Colab also makes it easy to work together on the same notebook. Sharing works much like sharing a Google Doc. Each notebook has **sharing settings** that control who can view or edit it. You can choose to share with specific people or allow access to anyone with the link.

14.5 5. Managing Python Packages

Python packages define what your notebook can do. In Colab, packages are installed into the current **runtime**, not permanently. This means installs are fast, but they disappear when the session resets.

A good habit is to make all package installs **explicit** and place them near the top of the notebook. This helps others run your code and helps you recover quickly after a reset.

14.5.1 Inspect installed packages

You can check which packages are already available in the current runtime.

```
# Shows all Python packages currently installed in this Colab runtime
%pip list
```

Colab includes common geospatial packages (e.g. gdal, geopandas, and xarray), but you should never rely on this without checking.

14.5.2 Installing packages with pip

In Colab, pip commands usually start with a **percent sign**. This tells the notebook to run the command in a way that is safe and consistent with the current Python runtime.

```
# Installs the required packages into the active runtime
%pip install geopandas rasterio leafmap
```

You may also see pip commands that start with an **exclamation mark**. This runs the command at the system level.

```
# Installs the package using a system command
!pip install pygis
```

Both approaches work in Colab. In this course, prefer the **percent pip** version because it avoids confusion when working with different runtimes. Do not use Conda in Colab. Always use pip.

14.5.3 Reducing output

Package installation can produce a lot of text. To keep notebooks readable, you can reduce the output (-q) --quiet .

```
# Installs the package while reducing console output
%pip install -q pygis
```

14.5.4 Check package versions

When code behaves differently than expected, checking the **version** is often the first step.

```
# Displays version number and installation path of the package
%pip show geemap
```

Different versions can lead to different results, especially in geospatial workflows.

14.5.5 Uninstall packages

If a package causes conflicts or you want to start fresh, you can remove it.

```
# Removes the package without asking for confirmation (-y = --yes)
%pip uninstall geemap -y
```

14.6 6. Working with Files and Data

Understanding where your data lives is essential when working in Colab. Files stored inside the Colab **session** exist only temporarily. If the session resets, those files are lost.

To avoid losing work, important files and results should be stored in **Google Drive**. Drive provides persistent storage that stays available across sessions and devices.

14.6.1 Using Google Drive for storage

Before accessing Drive, you need to mount it in your notebook.

```
# Connects your Google Drive to the Colab session
from google.colab import drive
drive.mount('/content/gdrive')
```

After mounting, your personal Drive is available under MyDrive. A good practice is to create a **course folder** and keep notebooks, data, and outputs organized there.

```
# Lists the contents of your Google Drive
import os
os.listdir('/content/gdrive/MyDrive')
```

14.6.2 Uploading and downloading data

You can upload files from your local computer directly into the current session. This is useful for small datasets or quick tests.

```
# Opens a file picker to upload files from your computer
from google.colab import files
uploaded = files.upload()
```

To download results back to your computer, use the following command.

```
# Downloads a file from the session to your local machine
from google.colab import files
files.download('results.csv')
```

Remember that uploaded files stored only in the session should be moved to Drive if you want to keep them.

14.6.3 Accessing data from the web

Many datasets can be accessed directly from the web. This avoids manual downloads and keeps workflows reproducible.

```
# Downloads a dataset from a public URL
!wget https://opengeos.org/data/world/world_cities.csv
```

Course code and example data are often stored on GitHub. You can clone a repository directly into your session.

```
# Copies a GitHub repository into the Colab session
!git clone https://github.com/giswqs/intro-gispro.git
```

Working directly with online sources makes it easier to rerun notebooks and share them with others.

14.7 8. Running Course Notebooks

All notebooks in this course can be run directly in **Google Colab**. This allows you to start working immediately without setting up a local environment.

When a page in this Jupyter Book provides an **Open in Colab** link, use it. The notebook will open in Colab and is ready to run. This is the recommended way to get started.

You can also open notebooks directly from the **GitHub repository**. Colab can load notebooks straight from GitHub without downloading anything.

Update the link!!! Example: Go to <https://colab.research.google.com/github/giswqs/intro-gispro/blob/main>

1. Open <https://colab.research.google.com/github/>
2. Navigate to the course repository
3. Select the notebook you want to run

Once the notebook is open, run the cells from top to bottom. This ensures that all **package installs**, **imports**, and **data loading** steps are executed in the correct order.

If the notebook modifies files or produces results, save a copy to **Google Drive**. This ensures your work persists even if the session resets.

Running notebooks in Colab is meant to support learning and experimentation. Feel free to change code, add cells, and explore, knowing that you can always reopen a clean version from the book or repository.

14.8 9. Exercises

The following exercises help you practice the learning objectives of this section. They focus on running notebooks in Colab, managing packages, and working with data in a session based environment.

14.8.0.1 Exercise 1: Colab setup

Objective

Get comfortable with Colab as a working environment for this course.

Tasks

1. Open Google Colab and create a new notebook.
2. Check which runtime is active and confirm that it uses Python.
3. Mount your **Google Drive**.
4. Install the required **Python packages** for this course.
5. Create a folder in your Drive for course notebooks and data.
6. Add a short text cell that explains what this notebook is for.
7. Save the notebook with a clear and descriptive name.

This exercise focuses on opening notebooks, installing packages, and using Drive for persistent storage.

14.8.0.2 Exercise 2: Working with files and data

Objective

Practice moving data between the Colab session, Google Drive, and the web.

Tasks

1. Download a small dataset from the web using `wget`.
2. Upload a second file from your local computer to Colab.
3. Move both datasets into your course folder in **Google Drive**.
4. Load one dataset into a Pandas DataFrame or a GeoPandas GeoDataFrame.

5. Inspect the data and print the first few rows.
6. Save a processed result to Drive.
7. Download the result back to your local computer.

This exercise reinforces how data lives in Colab sessions and how to make results persistent.

14.8.0.3 Exercise 3: Running and sharing a course notebook

Objective

Run a complete notebook and prepare it for collaboration.

Tasks

1. Open one course notebook in Colab using the provided link.
2. Run all cells from top to bottom without errors.
3. Modify one code cell in a meaningful way.
4. Add a text cell explaining what you changed and why.
5. Save your own copy to **Google Drive**.
6. Change the sharing settings so others can view the notebook.
7. Share the link with a peer and ask them to run it.

This exercise focuses on reproducible execution, documentation, and sharing notebooks with others.

15. JupyterLab

An interactive workspace for exploratory data analysis

15.1 1. Introduction

[JupyterLab](#) is the main **working environment** in this course. It is not just a place to run code, but a space where **thinking**, **coding**, and **documentation** come together.

In spatial data science, work is rarely linear. You load data, inspect it, try something, visualize the result, adjust your approach, and document what you learned. JupyterLab is designed for exactly this kind of **exploratory** and **iterative** work.

A Jupyter notebook combines **executable code**, **text explanations**, **figures**, and **maps** in one place. This makes it easier to:

- understand what your code does
- explain why you made certain choices
- revisit and reproduce your analysis later

This is especially important when working with spatial data, where data sources, projections, and processing steps can strongly influence results. By keeping code and explanation together, notebooks support **reproducibility**, **transparency**, and a clear **analysis narrative**.

In this section, you will learn how to use JupyterLab as a structured workspace for spatial data science. The focus is not on learning every feature, but on developing good **working habits** that you will use throughout the course and beyond.

This section focuses on **how we work**, not just **what we run**.

15.2 2. Learning Objectives

After working through this section, you should be able to:

- **explain** how JupyterLab fits into a spatial data science workflow
- **navigate** the JupyterLab interface and manage notebooks and files
- **create and run** notebooks that combine code, text, and visual output in a **reproducible** way

These objectives focus on using JupyterLab as a practical **working environment** for exploration, documentation, and analysis. You will use these skills throughout the course when developing and refining your own spatial data workflows.

15.3 3. Getting Started

Before using JupyterLab, it is important to think about **where** it is installed and **which environment** it belongs to. Most problems with JupyterLab do not come from the tool itself, but from using the **wrong Python environment**.

In this course, JupyterLab is always part of a **project specific environment**. This keeps your setup clean, avoids version conflicts, and makes your work easier to reproduce.

15.3.1 Recommended setup strategy

JupyterLab should **not** be installed system wide. Instead, you install it inside a dedicated **conda environment**.

The basic idea is simple:

- one **environment** per project

- the environment must be **activated** before starting JupyterLab
- all packages used in a notebook come from the active environment

If this is new to you, revisit the **Conda** section before continuing. Environment activation is a required habit for this course.

15.3.2 Installing JupyterLab

The recommended way to install JupyterLab for spatial data science is with **conda**, as it handles geospatial dependencies reliably. Install JupyterLab into the currently active conda environment (`-c = -channel`).

```
conda install -c conda-forge jupyterlab
```

If you are working in a pip based environment, you can install JupyterLab with:

```
pip install jupyterlab
```

Only use one package manager per environment. Mixing conda and pip without care is a common source of errors.

15.3.3 Verifying the installation

After installation, always check that JupyterLab is available in the **active environment**.

Run the following command in your terminal:

```
jupyter lab --version
```

You should see a version number printed in the terminal. If you get an error, check the following first:

- is the **correct environment** activated
- was JupyterLab installed into that environment
- are you accidentally using a different Python installation

Taking a minute to verify your setup now will save you a lot of time later.

15.4 4. Launching JupyterLab

Before working with notebooks, it helps to understand what actually happens when you start JupyterLab. This section focuses on building a **mental model** of how JupyterLab works, rather than on clicking through menus.

15.4.1 Run `jupyter lab`

When you run `jupyter lab`, you are not opening a normal desktop application. Instead, JupyterLab starts a **local server** on your computer.

Here is what is happening in the background:

- a **local server** starts and runs in your terminal
- your **web browser** acts as the user interface
- the address usually points to **localhost**, which means your own machine
- the terminal must stay open because it is running the server

If you close the terminal, the server stops and JupyterLab shuts down. Understanding this connection between **terminal**, **server**, and **browser** helps explain many common issues students run into later.

15.4.2 The JupyterLab interface

The JupyterLab interface is designed as a flexible workspace. A useful way to remember its structure is the following mental model:

Command – Canvas – Context – Status

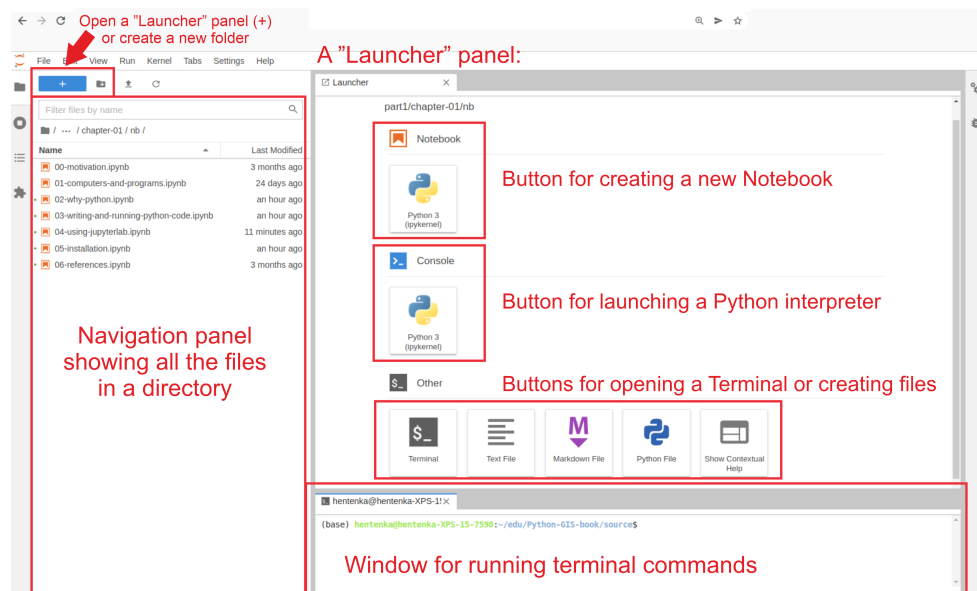


Figure 15.1: The main interface components of JupyterLab IDE (source: [python-gis-book](#)).

Each part has a clear role:

- **Left Sidebar**

This is your **project and control** area. You browse files, manage running notebooks, and access additional tools.

- **Main Area**

This is the **work canvas**. Notebooks, terminals, and files open here and can be arranged side by side.

- **Menu Bar**

This contains **global actions** such as saving files, running cells, changing kernels, and adjusting the layout.

- **Status Bar**

This provides **feedback and state** information, such as whether the kernel is busy or idle.

Keeping this structure in mind makes it easier to navigate JupyterLab and transfer these skills to other development environments later in the course.

15.5 5. Notebooks as Scientific Documents

In this course, notebooks are not temporary scratchpads. They are **scientific documents** that combine code, explanation, and results. A good notebook tells the story of your analysis in a way that others and your future self can understand.

15.5.1 Creating a notebook

When JupyterLab starts, you will see the **Launcher**. This is the entry point for creating new files.

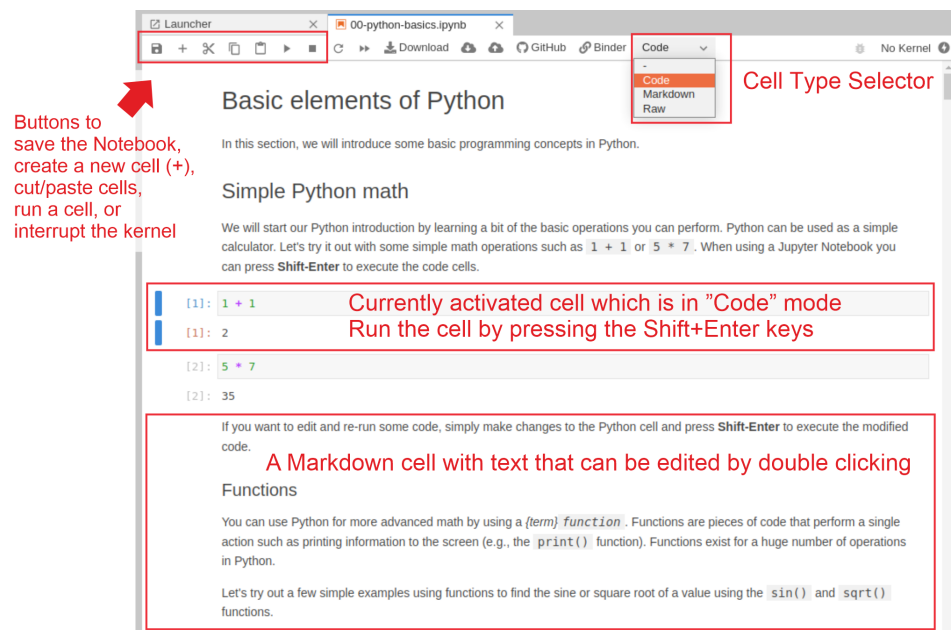


Figure 15.2: The basic components of a Jupyter Notebook opened in JupyterLab (source: [python-gis-book](#)).

To create a notebook:

- choose a **Python kernel** that belongs to your active environment
- create the notebook from the Launcher
- give the file a **clear and meaningful name**

Good naming matters. A notebook called `analysis.ipynb` tells you very little. A name like `snow_cover_exploration_v1.ipynb` makes the purpose clear even months later.

15.5.2 Cells, execution, and state

Notebooks run code in **cells**, and these cells are executed **sequentially**. This means the order in which you run cells matters.

A common source of confusion is **hidden state**. Variables can exist in memory even if the cell that created them is no longer visible or was changed later.

To avoid problems:

- run cells from top to bottom
- use **Restart Kernel and Run All** regularly
- check that your notebook works from a clean start

This habit is essential for **reproducibility** and for submitting reliable work.

15.5.3 The geospatial output

To see how notebooks behave, start with a minimal geospatial example. The goal here is not to understand the mapping library, but to observe the notebook workflow.

You typically go through three steps:

- **import** a library
- **create** an object
- display the **rendered output**

When you run a cell that produces visual output, JupyterLab automatically renders it below the cell. This tight link between code and output is what makes notebooks so powerful for **exploration** and **communication** in spatial data science.

15.6 6. Productivity Fundamentals

Working efficiently in JupyterLab is less about speed and more about **reducing cognitive load**. Using the keyboard helps you stay focused on your analysis instead of constantly switching between mouse and menus.

This section introduces only the **essential concepts and shortcuts** you need to work fluently. You do not need to memorize everything at once.

15.6.1 Notebook modes

Jupyter notebooks operate in two distinct modes. Many beginner mistakes happen because these modes are confused.

Edit mode

- used to *write and modify code or text*
- a cursor is visible inside the cell
- the cell border indicates active editing

Command mode

- used to *navigate and manage cells*
- no text cursor is visible
- the whole cell is selected as an object

Always be aware of which mode you are in. If something does not behave as expected, mode confusion is often the reason.

15.6.2 Core shortcuts

You only need a small set of shortcuts to work productively. These are the ones you will use constantly:

- **Shift Enter** to run a cell and move to the next one
- **Ctrl Enter** to run a cell and stay in place
- **Alt (⌘) Enter** to run a cell and create a new one below
- **Esc** to switch to command mode
- **Enter** to switch to edit mode
- **A** to insert a cell above
- **B** to insert a cell below
- **M** to change a cell to markdown
- **Y** to change a cell to code
- **DD** to delete a cell
- **Z** to undo a cell deletion

Most keyboard shortcuts depend on the **current notebook mode**. Shortcuts that act on the **structure of the notebook**, such as creating, deleting, or moving cells, only work in **command mode**. If they do nothing, you are usually still in edit mode. Shortcuts that **run cells**, such as Shift Enter or Ctrl Enter, work in both modes. Shortcuts that **edit text** only work in edit mode. Mastering these shortcuts already gives you most of the productivity benefits.

15.6.3 Editing vs Structuring

A useful way to think about notebook work is to separate two activities:

- **editing** content inside a cell which happens in edit mode
- **structuring** the analysis by adding moving or reorganizing cells which happens in command mode

Being aware of this distinction helps you work more deliberately. You edit when you think about code and text. You structure when you think about the **story** your analysis is telling.

15.7 7. Working Without a Local Setup

Not everyone can or wants to install software locally right away. For this reason, most parts of the course material can also be run using **Binder** and **Colab**.

Both services allow you to work with notebooks directly in your **browser**, without setting up a local Python environment.

15.7.1 What Binder and Colab are

Binder starts a temporary **JupyterLab** environment that is linked to a course repository. **Colab** provides hosted **Jupyter notebooks** that run on Google infrastructure.

Both options:

- run in the **browser**
- require no local installation
- are suitable for **learning and experimentation**

15.7.2 When to use them

Binder and Colab are useful when:

- you want to **quickly explore** course notebooks
- you are working on a **shared or restricted computer**
- your local setup is not ready yet

They help lower entry barriers and ensure everyone can participate.

15.7.3 Limitations to keep in mind

Both Binder and Colab have **limitations**:

- startup can take time
- environments are **temporary**
- file persistence is limited or requires extra steps

Because of this, they are best used for short tasks and exploration.

Binder and Colab are for **learning**, not for **long term projects**.

15.8 8. Exercises

These exercises help you practice using JupyterLab as a **working environment** for spatial data science. The goal is not to learn every feature, but to build confidence with the core workflows you will use throughout the course.

Work through the exercises at your own pace. If something breaks, that is part of the learning process.

15.8.0.1 Exercise 1: Setting Up Your JupyterLab Environment

Objective

Set up and verify a working JupyterLab environment for spatial data analysis.

What you practice

- working with **conda environments**
- installing packages
- launching and testing JupyterLab

Tasks

1. Create and activate a dedicated environment:

```
conda create -n geolab python=3.12
conda activate geolab
```

2. Install JupyterLab and basic geospatial packages:

```
conda install -c conda-forge jupyterlab geopandas matplotlib ipyleaflet
```

3. Launch JupyterLab:

```
jupyter lab
```

4. Explore the interface:

- locate the **file browser**, **main area**, **menu bar**, and **status bar**
- open multiple tabs and arrange them side by side

5. Verify your setup:

- create a new notebook
- run the following imports in a code cell:

```
import geopandas as gpd
import matplotlib.pyplot as plt
import ipyleaflet
print("Geospatial environment ready")
```

What to verify

- JupyterLab starts without errors
 - the notebook runs from top to bottom
 - all imports work successfully
-

15.8.0.2 Exercise 2: Keyboard Shortcuts and Efficient Workflow

Objective Develop fluency with keyboard driven notebook work.

What you practice

- switching between **edit** and **command** mode
- running and structuring cells
- reducing reliance on the mouse

Tasks

1. Practice essential shortcuts:
 - run cells with **Shift Enter**, **Ctrl Enter**, and **Alt Enter**
 - switch modes using **Esc** and **Enter**
 - create cells using **A** and **B**
 - delete and recover cells using **DD** and **Z**
2. Create a practice notebook using only the keyboard:
 - create at least **eight cells**
 - mix **code** and **markdown**
 - change cell types using **M** and **Y**
 - navigate using arrow keys in command mode
3. Practice editing and restructuring:
 - copy and paste cells
 - rearrange the order of cells
 - merge or split cells if needed
4. Check notebook state:

- restart the kernel
- run all cells from top to bottom
- confirm that the notebook still works

What to verify

- you can build and restructure a notebook without using the mouse
 - the notebook runs cleanly after a kernel restart
-

15.8.0.3 Optional Challenge: Workflow Efficiency

Objective Improve speed and confidence when working in notebooks.

What you practice

- combining shortcuts into a smooth workflow
- separating editing from structuring

Tasks

- create a new notebook
- add ten cells with alternating code and markdown
- use only the keyboard
- restart the kernel and run all cells

What to verify

- the notebook structure is clear
- all cells run without errors
- you feel more comfortable navigating and controlling JupyterLab

These exercises prepare you to use JupyterLab naturally in later labs and projects, where the focus will be on **spatial data analysis**, not on tool mechanics.

16. Motivation

Why programming matters

Welcome!

In this course, programming is not an end in itself.

It is a tool for **make sense of data**.

Over the past decades, the amount of data that is openly available has grown enormously. Climate records, satellite imagery, sensor networks, and administrative datasets can now be accessed within seconds. This creates exciting opportunities, but also a challenge: raw data are rarely meaningful on their own.

The central motivation of this course is simple:

How can we turn complex data into insight?

16.1 1. From data to understanding

Large datasets often appear as long tables of numbers, timestamps, or coded values. Even when the data contain important information, it is difficult to recognize patterns by looking at them directly.

Consider the first few lines of a climate data file:

```
029740 99999 195201010000 200 23 *** 15 0VC 7 2 * 5.0 63 ** ** ** ** ...
029740 99999 195201010600 220 18 *** 8 0VC 7 2 * 2.2 63 ** ** ** * ...
029740 99999 195201011200 220 21 *** 5 0VC 7 * * 3.8 59 ** ** ** * ...
...
```

At this stage, the data are technically correct, but cognitively overwhelming. Important questions remain unanswered:

- Is there a trend?
- How does the data change over time?
- Does space matter?

Programming allows us to bridge this gap.

16.2 2. Why visualization matters

One of the most powerful ways to understand data is visualization.

A simple plot of global temperature anomalies over time already reveals something that is nearly impossible to see in the raw text: long-term warming and changes in variability. A single figure can compress decades of measurements into an immediately interpretable form.

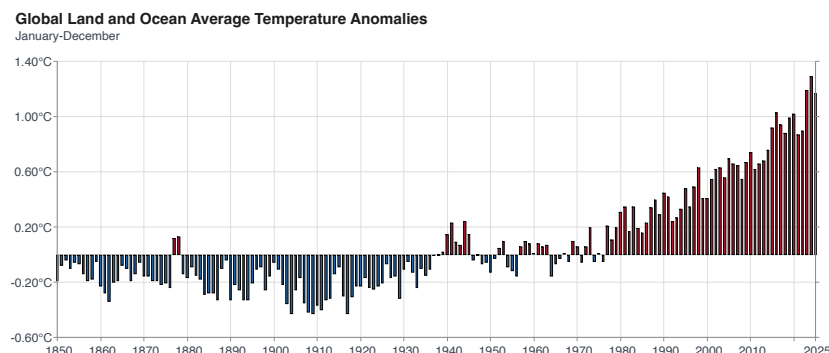


Figure 16.1: Figure from NOAA [Global Climate Report](#) - Annual 2025.

Maps add another layer of understanding. When the same data are shown spatially, we begin to see regional differences and geographic patterns. Space suddenly becomes part of the story.

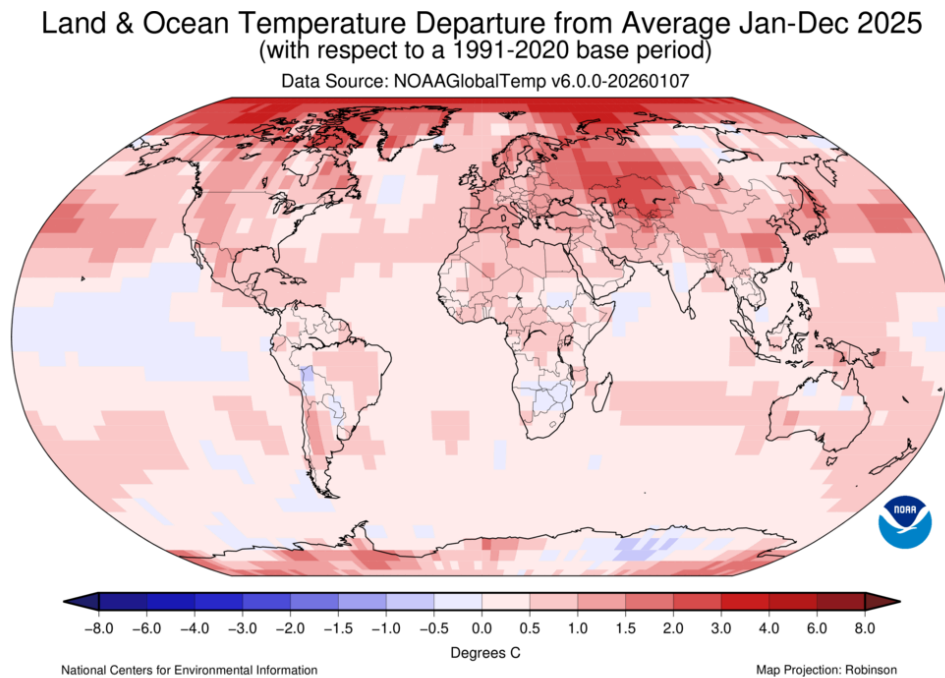


Figure 16.2: Figure from NOAA [Global Climate Report](#) - Annual 2025.

These visual representations do not change the data. They change **how we understand it**.

16.3 3. Space and time together

Many of the most interesting questions in geography and environmental science involve **both space and time**.

Static maps show where something happens. Time series show how something changes. Combining both reveals dynamics.

Animated visualizations, such as global temperature anomaly animations, allow us to follow processes as they unfold across the Earth. Patterns that were previously abstract become intuitive. Viewers can often explain what they see without needing technical background knowledge.

What makes this particularly powerful is that such visualizations are not created by hand. They are produced through **reproducible code**, which means anyone can inspect the steps, rerun the analysis, and verify the result.

16.4 4. Why programming helps

Programming allows us to:

- automate complex workflows
- scale analysis to large datasets
- document every analytical step
- make results transparent and reproducible

Even basic programming skills already make a difference. A few lines of code can load thousands of measurements, summarize them, and produce a clear visual result.

You do not need to become an expert programmer to benefit. The focus of this course is **applied programming for data understanding**.

16.5 5. What this course aims to do

Throughout this course, you will learn how to use Python to:

- read and explore data
- transform raw values into meaningful summaries
- visualize patterns in space and time
- communicate results clearly

The visual examples shown here are not goals in themselves. They are inspiration for what becomes possible once data, code, and interpretation come together.

By the end of this part of the course, you will not just run code. You will use code to **ask and answer questions**.

16.6 6. Summary

In this section, you explored **why programming matters**.

You have seen that:

- Raw data are often difficult to interpret directly
- Visualization can reveal patterns that remain otherwise hidden
- Reproducible code allows analyses to be transparent and repeatable

Programming is about transforming data into insight.

The key message is this:

Code is not just a technical skill.
It is a structured way of thinking about data.

16.6.1 Looking ahead

So far, we have focused on **why** programming matters.

In the next section, you will learn:

- what programming means in this course
- why we use Python
- how we run code using [notebooks](#)

We now move from motivation to method.

17. Glossary